

TOUR.md — Lộ trình nắm rõ CLeak từ đầu tới cuối

Cách dùng tài liệu này. Đây là lộ trình học chia theo các buổi, viết cho chính tác giả hệ thống trước khi bảo vệ luận văn. Mỗi phần (P0–P9, kèm P8.5) là một buổi học gồm ba việc: (a) đọc phần đó, (b) mở đúng các file và dòng được trích dẫn để đối chiếu code, (c) trả lời hết mục **Tự kiểm** ở cuối phần. Mọi con số và vị trí **file:dòng** đều được đối chiếu trực tiếp với code trên máy tại thời điểm viết (commit **1ce6e97** trở đi). Lý do phải đối chiếu code: tài liệu mô tả code sẽ sai ngay khi code thay đổi; tài liệu trở đúng chỗ code nằm thì chỉ cần kiểm tra một cái nhìn là biết tài liệu còn đúng hay không. Nếu code đổi sau này, hãy sửa đúng phần TOUR tương ứng — danh sách mục ở đầu file chính là bản đồ để bảo trì chính tài liệu này.

Về từ ngữ: thuật ngữ chuyên ngành (MCP, LLM, token, cache, bundle, verdict, corpus, baseline...) được giữ nguyên vì đó là tên gọi chính thức trong code và trong tài liệu để bài; phần còn lại viết bằng tiếng Việt đơn giản.

Danh sách các buổi học

- ☒ **P0** — Bản đồ tổng thể và cách tư duy đúng (30–45 phút)
- ☒ **P1** — Ba lối vào hệ thống + cấu hình + hệ thống sự kiện (1 buổi)
- ☒ **P2** — **runScan**: trực chính 7 giai đoạn (1 buổi) — *quan trọng nhất*
- ☒ **P3** — Stage A: sub-agent thu thập bằng chứng + chi phí token (1 buổi)
- ☒ **P4** — **packages/agent-core**: vòng lặp gọi công cụ, callModel, compaction (1–2 buổi)
- ☒ **P5** — Bên trong static-analyzer: CParserService + 11 công cụ (1 buổi)
- ☒ **P6** — dynamic-analyzer + mô hình tin cậy bảo mật (1 buổi)
- ☒ **P7** — Tầng judge + mô hình dữ liệu **@cleak/common** (1 buổi)
- ☒ **P8** — Khung đánh giá: corpus gate, determinism gate, ánh xạ FREEZE (1 buổi)
- ☒ **P8.5** — Dữ liệu đánh giá: 3 corpus đến từ đâu, dựng thế nào, chạy thế nào (1 buổi)
- ☒ **P9** — Chuẩn bị bảo vệ: 20 câu hỏi + cách kể câu chuyện số liệu (1 buổi)

Vì sao học theo thứ tự này

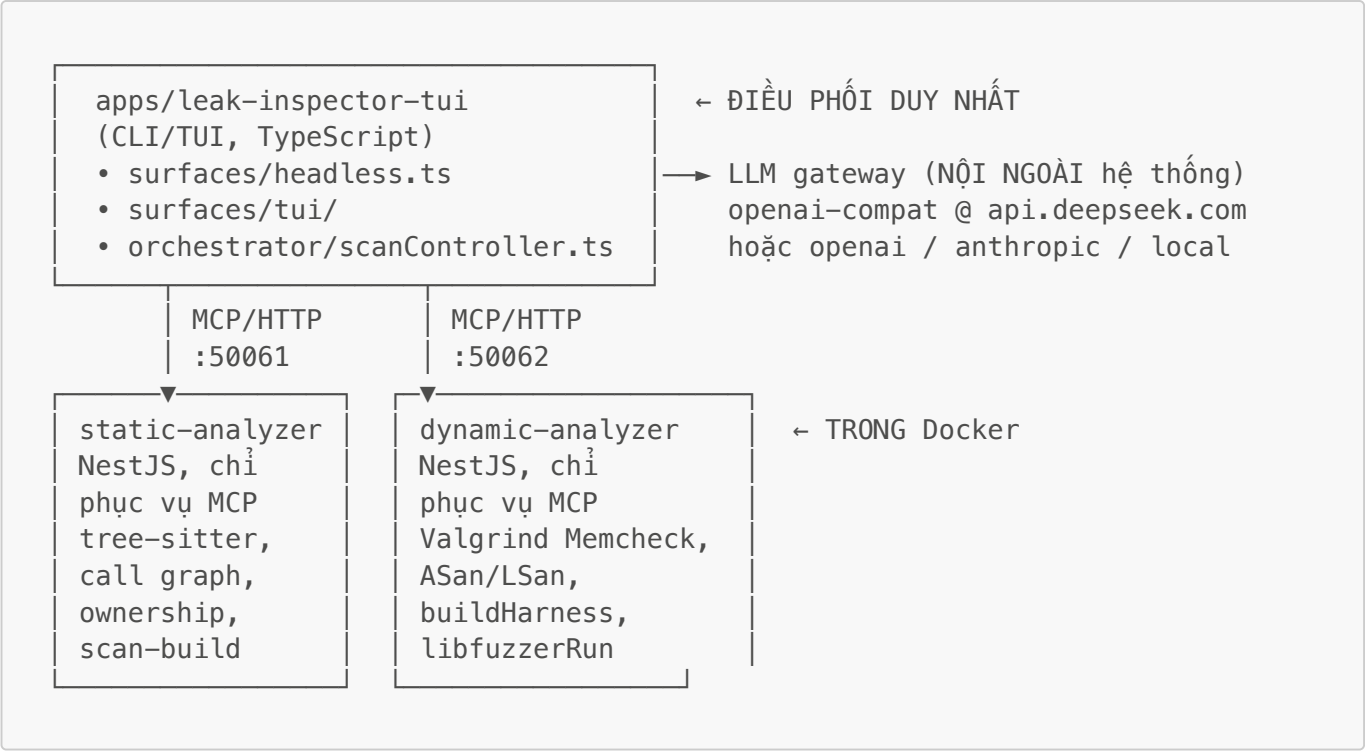
Thứ tự P0 → P9 đi **từ ngoài vào trong**: trước tiên nhìn toàn cảnh (P0), rồi xem người dùng đi vào hệ từ đâu (P1), sau đó đọc trực chính mà mọi thứ đều đi qua (P2), rồi lần lượt mở từng bộ phận (P3–P7), cuối cùng là nơi sinh ra số liệu (P8, P8.5) và chuẩn bị bảo vệ (P9). Lý do sắp như vậy: không nắm được "ai gọi ai" thì đọc code bên trong sẽ không biết mình đang đọc để trả lời câu hỏi nào. Trong tất cả các phần, **P2 là trực chính** — mọi giai đoạn khác chỉ là "mở rộng một phần của P2", nên nếu chỉ có thời gian học một buổi, hãy học P2.

P0 — Bản đồ tổng thể và cách tư duy đúng (30–45 phút)

Mục tiêu

Nhìn được **toàn bộ hệ thống trong một trang**: ai đứng ở đâu, dữ liệu chảy theo đường nào, và — quan trọng nhất khi bảo vệ — hiểu ba quyết định thiết kế làm nên mọi thứ còn lại. Ba quyết định này cần hiểu trước khi đọc code, vì chúng trả lời câu "vì sao code viết như vậy" cho phần lớn các phần sau.

1. Sơ đồ các tiến trình chạy thật



Nguồn: docs/ARCHITECTURE.md:49 (bảng giao thức), apps/leak-inspector-tui/src/surfaces/headless.ts:10-22 (các lệnh import cho thấy chính xác các bộ phận này được nối với nhau thế nào).

Ba đặc điểm của sơ đồ căn thuộc lòng — và vì sao:

1. **Chỉ có một đường điều phối duy nhất.** Không có giao diện web, không có gRPC, không có "chế độ B". Mọi lệnh quét — chạy từ dòng lệnh, chạy trong TUI, hay chạy đánh giá theo lô — đều đi qua đúng một hàm: `runScan()`. Vì sao quy định như vậy: khi mọi con đường hội tụ về một chỗ, việc sửa một hành vi chỉ cần làm một lần và chắc chắn áp dụng cho cả ba lối vào; nếu có nhiều đường song song thì mỗi đường phải sửa riêng và rất dễ lệch nhau. (Lịch sử: đường web cũ đã bị xóa khỏi `master`, còn lưu trên nhánh `web-implementation`; gRPC và thư mục `proto/` cũng đã xóa vì sau khi web bị bỏ thì không còn ai gọi gRPC nữa.)
2. **LLM nằm NGOÀI biên hệ thống.** LLM chỉ là một dịch vụ HTTP từ bên ngoài (kiểu OpenAI-compatible). Hệ không nhúng model nào vào code; muốn đổi model chỉ cần đổi file cấu hình `~/.config/cleak/config.json`. Vì sao quan trọng: toàn bộ tính "không lặp lại được" (non-determinism) của chế độ `llm_assisted` đến từ đúng một điểm — vòng gọi `callModel` trong `agent-core`. Biết điểm nào là nguồn không-lặp-lại giúp cô lập và kiểm soát nó thay vì lo lắng rải rác khắp nơi.
3. **Hai analyzer là dịch vụ thuần MCP.** Mỗi analyzer là một ứng dụng NestJS mà `main.ts` chỉ làm đúng một việc: dựng DI context rồi mở cổng MCP/HTTP. Không REST, không gRPC. Danh sách công cụ (tool) và định dạng đầu vào khai báo bằng Zod ngay trong `apps/static-analyzer/src/mcp/static-mcp-server.ts` và `apps/dynamic-analyzer/src/mcp/dynamic-mcp-server.ts`. Vì sao dùng MCP: đây là giao thức chuẩn cho việc agent gọi công cụ, và hai analyzer cần chạy trong Docker tách biệt với trình điều phối vì chúng phải chạy code không tin tưởng (P6).

2. Vai trò từng package — đọc một lần là nhớ

Đường dẫn	Vai trò	Một câu chốt
apps/leak-inspector-tui	Điều phối	Điều phối 4 stage, viết báo cáo, giữ mọi quyết định luồng công việc

Đường dẫn	Vai trò	Một câu chốt
<code>packages/agent-core</code>	Bộ máy agentic	Vòng lặp gọi công cụ tự viết (không dùng framework), MCP client, <code>callModel</code> đa nhà cung cấp, compaction
<code>packages/common</code>	Ngôn ngữ chung	<code>LeakBundle</code> , Zod schemas, heuristic judge + LLM/consensus judge, các trình dựng báo cáo — "một file kiểu dữ liệu mà mọi thứ đều import"
<code>packages/config</code>	Nguồn cấu hình	Zod schema + nạp/lưu JSON tại <code>~/.config/cleak/config.json</code> ; thứ tự ưu tiên: tham số dòng lệnh > file cấu hình > giá trị mặc định
<code>packages/observability</code>	Log có cấu trúc	JSON logging, dùng xuyên suốt
<code>apps/static-analyzer</code>	Bộ phận phân tích tĩnh	11 công cụ MCP, mọi phân tích đi qua <code>CParserService</code>
<code>apps/dynamic-analyzer</code>	Bộ phận phân tích động	11 công cụ MCP, công thức build + sanitizer

3. Ba quyết định thiết kế định hình tất cả

Quyết định 1 — Agentic nhưng có rào chắn. Hệ là agent, nghĩa là LLM tự chọn công cụ tiếp theo cần gọi. Nhưng có hai điểm được "khóa cứng" để kết quả không phụ thuộc tâm trạng của model: (a) giai đoạn phân tích động (Stage B) chạy theo công thức cố định do code điều phối, không phải do LLM quyết; (b) bằng chứng mà Stage A (các sub-agent LLM) thu được được **chép vào một kho có cấu trúc** (`staticStore`) mỗi lần LLM gọi công cụ — phần văn bản tự do mà LLM viết ra không được ghi vào đâu cả. Vì sao cần rào như vậy: nếu LLM được quyết định cả việc chạy gì lẫn việc ghi gì, thì hai lần chạy có thể cho hai bộ bằng chứng khác nhau, và khi đó không thể phân biệt "chênh lệch do model" với "chênh lệch do quy trình". Nhờ khóa hai điểm này, khác biệt giữa `no_llm` và `llm_assisted` nằm gọn trong các điểm đã biết — đó là lý do các so sánh A/B trong luận văn đáng tin.

Quyết định 2 — Judge lai 3 tầng. Heuristic (bộ quy tắc tính điểm, không dùng LLM) chấm **tất cả** bundle vì nó rẻ và luôn cho cùng kết quả; LLM judge chỉ được gọi với các bundle **borderline** (nằm trong dải mơ hồ) vì gọi LLM cho mọi bundle vừa đắt vừa không cần thiết; consensus (lấy k mẫu LLM rồi gộp) là lựa chọn thêm, không mặc định. Hệ quả đo được trên MemHint: mỗi run có khoảng 17.1K phán quyết do heuristic xử lý, LLM judge chỉ thực sự chạy 2 lần trong 3 run — vì corpus này hầu như không có bundle nào borderline nên `llm_assisted` cho kết quả y hệt `no_llm`. Hiểu quyết định này là hiểu được cả kết quả MemHint lẫn chuyện vì sao tiêu 44.3M token mà điểm số không đổi.

Quyết định 3 — Tính lặp lại là công chặn, không phải lời hứa. Chế độ `no_llm` bắt buộc phải cho kết quả **trùng từng byte** giữa hai lần chạy, và điều này được CI kiểm tra (`scripts/determinism-gate.sh`), trong đó gate này chủ động loại bỏ 2 cách "đạt pass giả" từng gặp thật trong quá trình phát triển: so sánh kết quả với chính nó (do timestamp trùng nhau) và run toàn lỗi (hai run cùng toàn lỗi thì "giống nhau" nhưng vô nghĩa). Chế độ `llm_assisted` thì không thể trùng từng byte (có LLM mẫu ngẫu nhiên), nên báo cáo theo phân phối (chạy `--runs N` lần, lấy trung bình ± độ lệch chuẩn). Corpus phải qua khóa nội dung (`checkCorpusGate()`) trước khi được phép đo — vì một con số chỉ đáng tin khi biết chính xác nó đo trên dữ liệu nào. Cụm "two-tier reproducibility protocol" này là đóng góp số 2 của luận văn.

Tự kiểm P0

1. Vẽ lại sơ đồ các tiến trình từ trí nhớ, đúng cổng và đúng ba loại đường (vòng gọi công cụ → LLM; MCP → analyzer; ghi file → results/).

2. Một quét chạy qua TUI và một quét qua CLI: khác nhau ở đâu, hội tụ ở điểm nào? (Khác ở lối vào; hội tụ tại `runScan` — vì cả ba lối vào đều dựng cùng một bộ phụ thuộc rồi gọi hàm này.)

3. Vì sao gRPC bị xóa mà MCP được giữ? (Vì sau khi bỏ đường web, không còn ai gọi gRPC; MCP là giao thức duy nhất mà trình điều phối dùng để gọi analyzer.)

4. Điểm nào trong hệ chịu toàn bộ tính không-lặp-lại của `llm_assisted`? (Vòng gọi LLM trong `agent-core` — Stage B và việc ghi bằng chứng đều đã bị khoá thành cố định.)

5. Hybrid judge: heuristic xử lý bao nhiêu phần trăm phán quyết trên MemHint? Trên Juliet đầy đủ? (`{heuristic:17154, llm:2}` tức ~99.99%; Juliet `llm_assisted` toàn corpus: `heuristic 1826/2749 = 66.4%`, `llm 923 = 33.6%`.)

P1 — Ba lối vào + cấu hình + hệ thống sự kiện (1 buổi)

Mục tiêu

Nắm "hành lang cửa" của hệ: người dùng và các script đi vào từ đâu, cấu hình được đọc theo thứ tự ưu tiên nào, và mọi thứ báo cáo tiến trình qua cơ chế sự kiện ra sao. Học xong P1 bạn có thể chạy bất kỳ lệnh nào của hệ và hiểu được từng dòng đầu ra — vì mọi dòng output đều sinh ra từ các cơ chế trong phần này.

1. Ba lối vào — ba cửa, một điểm hẹn

Lối vào	Điểm vào	Lệnh	Dùng khi
Headless	<code>apps/leak-inspector-tui/src/surfaces/headless.ts</code>	<code>pnpm exec tsx src/cli.ts scan -- repo <path> / eval -- corpus <path> / tools</code>	Chạy có khả năng lặp lại — toàn bộ số liệu của luận văn đo từ đây
TUI	<code>apps/leak-inspector-tui/src/surfaces/tui/</code> (screens, hooks)	<code>pnpm run dev (cli.ts tui)</code>	Tương tác: <code>/scan</code> , <code>/config</code> , <code>/report</code> , <code>/tools</code>
Eval CLI	<code>evaluation/cli.ts</code> + <code>configs/baselines/B*.yaml</code>	<code>pnpm run eval:corpus</code> , wizard, quét B1–B7	Chạy so sánh cấu hình, lấy mẫu stratified, quét baseline

Điểm hẹn chung: **cả ba lối vào đều dựng cùng một bộ phụ thuộc rồi gọi `runScan()`** (`scanController.ts:646`) — 4 nơi gọi `runScan` nằm ở `surfaces/headless.ts` và `surfaces/tui/runner.ts`. Vì sao hội tụ: để ba cách dùng cho **cùng một hành vi** — nếu TUI quét khác CLI thì số liệu đo bằng CLI sẽ không mô tả được sản phẩm người dùng chạy.

Chi tiết headless đáng nhớ (`headless.ts`):

- Dựng chuỗi khởi động: `McpClient` + `buildCallModel` (từ `agent-core`) → `loadConfig` → `buildPathResolver` → `ScanEmitter` với các sink (`Jsonl/Multi/Callback`) → `buildWorkflowInvestigationPhase` → `runScan`.

- Có bước **allocator profiling** trước khi quét: `loadOrProfileAllocators`
 - `verifyAllocatorProfile` (`domain/allocatorProfiler.ts`, `domain/allocatorVerification.ts`). Việc này làm gì: mỗi dự án thật có các hàm cấp phát riêng (ví dụ tmux dùng `xmalloc`, redis dùng `zmalloc` thay vì `malloc` gốc) — nếu không nhận diện được thì mọi cấp phát qua hàm bọc đều vô hình với quét tĩnh. Kết quả profiling được cache theo từng profile và truyền vào các công cụ qua tham số `extraAllocators/extraDeallocators` — đó là lý do `candidateScan/callGraph/interproceduralFlow` có tham số này.
- `makeScanId` (`headless.ts:493`): `scan_<repo>_<YYYYMMDD-HHMMSS>_<hash6>` — tên thư mục kết quả `results/<scanId>/` sinh từ tên repo + thời điểm + mã băm, để hai lần quét cùng repo khác thời điểm không đè lên nhau.

Chi tiết TUI đáng nhớ (`surfaces/tui/hooks/useCommands.ts`):

- `/tools` chạy 3 phép kiểm tra kết nối **song song** (`useCommands.ts:452`): ping LLM (gửi câu "Respond with exactly one word: OK" qua đúng cấu hình provider — `testLlmResult:455`), ping MCP tĩnh, ping MCP động. Chạy song song vì ba phép kiểm độc lập nhau, chạy tuần tự chỉ làm người dùng chờ lâu hơn.
- `/report` không đối số → tìm thư mục `scan_*` mới nhất trong results theo thời gian sửa (`mostRecentScanId:399`).
- Preflight có thể chạy độc lập trước khi quét (`doPreflight:408`).

2. Cấu hình — một file JSON duy nhất

- Đường dẫn: `~/.config/cleak/config.json`. **TUI/CLI không dùng file .env** — chỉ các dịch vụ analyzer trong Docker dùng `.env` cho biến môi trường. Vì sao tách: cấu hình TUI là lựa chọn cá nhân (model nào, endpoint nào) còn `.env` của Docker là cấu hình môi trường chạy (đường dẫn, cổng) — hai thứ này đổi vì lý do khác nhau, gộp lại sẽ khó chuyển máy.
- Thứ tự ưu tiên: **tham số dòng lệnh > file cấu hình > mặc định** — kiểm tra bằng `cleak config get` (in ra cấu hình đã gộp).
- Schema + bộ nạp: `packages/config (@cleak/config)` — Zod schema, hàm `loadConfig`, `toProviderSettings` (biến cấu hình thành tham số cho `buildCallModel`), các lệnh `config init/get/set/unset`.
- Provider theo **profile**: ngoài 4 loại có sẵn (`local/openai/anthropic/ openai-compatible`), bất kỳ tên nào cũng dùng làm profile nếu khai `provider` chỉ định kiểu giao thông — ví dụ profile `deepseek-direct` trở `baseUrl https://api.deepseek.com/v1`. Lịch sử model của hệ (cần nhớ cho câu hỏi về model): `local/mimo-v2.5-pro` (thí nghiệm 2026-06) → `deepseek-v4-flash-0731` (các run đóng băng, `openai-compatible`) — xem `docs/EVALUATION.md:158-161`, cột Model trong `docs/RESULTS-FREEZE.md`.
- **Bẫy kinh điển**: cấu hình có khóa `workflow.targetedHarness.enabled` (Stage B2, mặc định **tắt**) và `configStaticEnrich` — đừng nhầm với các cờ per-run trên dòng lệnh. Đây là những thiết lập "ẩn" thay đổi hành vi giai đoạn mà không cần sửa code — vì vậy khi so sánh hai lần chạy, phải so sánh cả file cấu hình chứ không chỉ dòng lệnh.

3. Hệ thống sự kiện — nhịp tim hiển thị

- Lỗi: `orchestrator/events.ts` — `ScanEmitter` + `ScanEventName` + các sink (`JsonlFileSink`, `MultiSink`, `CallbackSink`, `EventSink`).
- `runScan` phát sự kiện đúng theo từng giai đoạn (xem lại trình tự ở P2): `SCAN_CREATED` → (`WORKSPACE_STARTED/FINISHED`, `BUILD_PLAN_SELECTED`) → `DYNAMIC_STARTED/FINISHED` →

JUDGING_STARTED/FINISHED → REPORTING_STARTED/FINISHED → COMPLETED — sự kiện COMPLETED mang {candidates, confirmed, likely} (scanController.ts:637-641).

- Sink là cách hệ xuất một luồng sự kiện ra nhiều nơi: headless ghi JSONL vào thư mục kết quả, TUI vẽ lên màn hình, eval CLI thu qua CallbackSink — cùng một luồng, ba cách tiêu thụ. Đây là mẫu thiết kế observer thuần túy; lợi ích thực dụng: khi một lần quét lạ cần kiểm tra, mở file JSONL của scanId là có dòng thời gian đầy đủ mà không phải chạy lại.
- Sự kiện từ agent (onAgentEvent, scanController.ts:574) là kênh riêng cho tin từ sub-agent/LLM (kiểu notice) — đừng lẫn với ScanEventName (kênh theo giai đoạn). Vì sao tách: tin giai đoạn dùng để vẽ tiến trình tổng, tin agent dùng để theo dõi từng agent riêng — gộp một kênh sẽ không lọc được "đang xảy ra ở ai".

Tự kiểm P1

1. Số liệu trong luận văn đo từ lỗi vào nào? Vì sao không dùng TUI? (Headless — vì TUI phụ thuộc terminal hiển thị, còn headless ghi JSONL thuần, kết quả lặp lại được.)
2. cleak config set provider deepseek-direct — thứ tự ưu tiên nào quyết giá trị cuối? Kiểm tra bằng gì mà không cần mở file? (cleak config get.)
3. Allocator profiling xảy ra trước hay sau runScan? Phục vụ công cụ nào? (Trước — trong lúc dựng headless; phục vụ các công cụ tĩnh nhận extraAllocators/extraDeallocators.)
4. Một lần quét treo ở giai đoạn động — mở gì để xem dòng thời gian? (File JSONL của scanId trong results/.)
5. Tin notice từ agent đi qua kênh nào? (onAgentEvent — kênh agent-event riêng, không phải ScanEventName.)

P2 — runScan: trực chính 7 giai đoạn (1 buổi)

Mục tiêu

Đọc hiểu toàn bộ vòng đời một lần quét đúng theo code: mỗi giai đoạn đọc và ghi gì vào bundle nào, và nắm 3 quy tắc bất biến (cache theo từng lần quét, cửa chặn cho giai đoạn động, chạy song song có kiểm soát) mà mọi số liệu lặp lại được của luận văn dựa vào. Sau buổi này bạn phải mở được scanController.ts và tự đọc không cần ai giải thích.

File trung tâm: apps/leak-inspector-tui/src/orchestrator/scanController.ts (một file khoảng 700 dòng chứa toàn bộ bộ điều phối). Nguyên tắc đọc: TOUR chỉ đường, code là chân lý — vì TOUR có thể lỗi thời nhưng code trên máy thì không.

1. Bản đồ file scanController.ts — mở lên là thấy ngay

Dòng	Ký hiệu	Vai trò
55	JULIET_SUPPORT_FILES	Tập file hỗ trợ Juliet bị loại khỏi discovery
63	ScanInput	Đầu vào quét: repoPath, analysisMode, dynamicMode, buildCommand, enrich, staticTools, fileLimit, extraAllocators/Deallocators, evalStaticPathMap...
91	ScanDeps	Các phụ thuộc tiêm vào: emitter, staticClient, dynamicClient?, investigation?, now?, onAgentEvent...

Dòng	Ký hiệu	Vai trò
126/141	<code>analyzerPath/hostPath</code>	Dịch đường dẫn giữa máy chủ và analyzer qua <code>evalStaticPathMap</code>
170	<code>enrichStaticEvidence</code>	Làm giàu bằng chứng tĩnh theo công thức cố định (chi tiết §4)
265	<code>runPreflight</code>	Ping static client (PREFLIGHT_STARTED/FINISHED) — fail nhanh
279	<code>runDiscovery</code>	Quét repo → candidates → bundles (chi tiết §3)
498	<code>runEnrichment</code>	Cửa chặn cho enrichment (chi tiết §4)
516	<code>runInvestigation</code>	Stage A agentic, chỉ chạy khi <code>llm_assisted</code> (chi tiết §5)
558	<code>runDynamicStage</code>	Công thức động cố định cho <code>no_llm</code> (chi tiết §4)
592	<code>runJudging</code>	Bộ chốt heuristic (chi tiết §6)
606	<code>runReporting</code>	buildReport + COMPLETED (chi tiết §7)
646	<code>runScan</code>	Điểm hẹn cả 7 giai đoạn

2. `runScan` (:646-685) — đọc theo thứ tự dòng

```
export async function runScan(input, deps): Promise<ScanResult> {
  const now = deps.now ?? (() => new Date().toISOString()); // :648
  const candidates = new CandidateManager(now); // :650
  const caches = createScanCaches(); // :654
  emitter.emit(SCAN_CREATED, {scanId, repoPath, mode}); // :656
  await runPreflight(staticClient, emitter); // :658
  // workspace (đường dẫn máy chủ, BUILD_PLAN_SELECTED nếu có buildCommand) //
:661-663
  const { dynamicRanInDiscovery } = await runDiscovery(...); // :665
  const enrichmentPromise = runEnrichment(...); // :676
  const dynamicPromise = runDynamicStage(...); // :677
  await Promise.all([enrichmentPromise, dynamicPromise]); // :678
  const outcome = await runInvestigation(deps, input, candidates,
dynamicRanInDiscovery, caches); // :680
  await runJudging(emitter, candidates.getAllBundles(), outcome); // :681
  const { report } = await runReporting(...); // :682
  return { report, bundles, investigation }; // :684
}
```

Ba chi tiết cần nhớ và lý do của chúng:

- now có thể thay thế được** (:648): khi test và đánh giá, người ta truyền vào một đồng hồ cố định. Vì sao quan trọng: mỗi bundle ghi `createdAt/updatedAt`, nếu dùng giờ thật thì hai lần chạy `no_llm` sẽ khác nhau ngay ở dấu thời gian — phá vỡ cam kết trùng từng byte. Đồng hồ cố định là một trong những mắt xích giúp `no_llm` trùng từng byte giữa hai lần chạy.
- createScanCaches()** (:654, file `domain/scanCaches.ts`): bộ hai cache — `files` (nội dung file: mỗi file đọc đĩa **1 lần mỗi quét**) và `tools` (kết quả MCP, khóa theo **tên công cụ + tham số (trừ content)**: câu hỏi lặp lại không phải trả tiền token lần hai). Quy tắc phạm vi được nêu rõ trong comment đầu file: cache tạo mới trong `runScan`, truyền theo tham chiếu xuyên suốt, hủy khi hàm trả về — **không có gì ở**

cấp module, vì vậy không thể có nội dung cũ của lần quét trước rò sang lần quét sau. Vì sao khắt khe vậy: một cache chéo-lần-quét sẽ làm hai lần quét "giống nhau" một cách giả tạo — trúng đúng cái gate determinism muốn bắt.

- **Cache được truyền cả vào investigation** (:543-546): khi sub-agent Stage A hỏi lại câu hỏi mà enrichment đã hỏi, hệ trả từ cache thay vì gọi lại MCP. Đây là ý nghĩa của comment "P0-1/P0-2: share the scan's memo caches".

3. Giai đoạn Discovery — **runDiscovery** (:279-495)

Công việc: liệt kê file `.c/.h/.cpp...` của repo → gọi **candidateScan** (quét từ vựng + AST) cho từng file → **normalizeCandidate()** đổi tên trường từ camelCase sang snake_case (**candidateState.ts:69**) → nạp vào **CandidateManager**. Có hai đường:

- **Discovery tĩnh** (mặc định, **staticDiscovery !== false**): quét candidates từ các vị trí cấp phát bộ nhớ.
- **Discovery chỉ-dynamic** (**static=false**): gọi **runDynamicOnlyDiscovery** (**domain/dynamicDiscovery.ts:67**, 2 nơi gọi trong scanController) — ****build**
 - chạy chương trình ngay trong **discovery****, gắn coverage lên bundle, trả **dynamicRanInDiscovery=true** để các giai đoạn sau không build lại lần nữa.

CandidateManager (**domain/candidateState.ts:12-52**) — một Map trong bộ nhớ **bundleId** → **LeakBundle**, chỉ 4 hàm: **ingest** (trùng bundleId thì trả về bundle cũ — chống nhân đôi), **getBundle**, **getAllBundles**, **attachEvidence** (thêm bằng chứng + cập nhật **updatedAt**).

Câu chuyện lỗi cần thuộc — **computeBundleId** (**candidateState.ts:54-66**): **id = bundle_ + SHA1 của allocation_site || file:line**. Phiên bản cũ lấy chỉ 6 ký tự đầu + 10 ký tự cuối của chuỗi — với mọi khóa dài hơn 16 ký tự (và khóa thật nào cũng vậy: đường dẫn đầy đủ + dòng + tên hàm cấp phát), các candidates **cùng phần đầu đường dẫn và cùng phần cuối tên allocator** (ví dụ mọi lệnh **curlx_malloc** của cả codebase) rơi trùng bundleId: trên **curl_1098e104**, **622 candidate thô chỉ còn lại 65 bundle**. Cách sửa: băm SHA1 toàn bộ khóa. Vì sao câu chuyện này đáng nhớ: nó là ví dụ sống động nhất cho câu hỏi "dữ liệu đánh giá có thể giấu lỗi một cách im lặng kiểu nào" — lỗi không làm hệ báo đỏ, chỉ làm số liệu sai mà không ai để ý.

LeakBundle (**packages/common/src/types/candidate.ts:16**): **candidate + evidence[]** (bằng chứng chung) + **staticEvidence?** (bằng chứng tĩnh có cấu trúc: ownership, cặp alloc→free, đường lộ rò — do enrichment ghi vào) + **dynamicCoverage?** (do giai đoạn động ghi vào) + **verdict? + status** (PENDING→...). Hai trường có dấu **?** vì lý do tương thích phiên bản: báo cáo cũ được ghi ra trước khi trường tồn tại vẫn phải đọc được.

4. Giai đoạn song song — Enrichment cùng lúc với Dynamic (:676-678)

Enrichment (ghi vào **staticEvidence**) và dynamic (ghi vào **evidence/ dynamicCoverage**) **chạm vào các trường khác nhau** của bundle, nên chạy **Promise.all** cùng lúc không gây ghi đè lẫn nhau — đây là tiền đề của việc song song hóa. Mỗi pipeline đều tự bỏ qua khi cửa chặn tắt, nên hành vi mặc định không đổi.

4a. **runEnrichment** (:498-513) — cửa chặn

```
enrichOn = staticDiscovery && (input.enrich ?? configStaticEnrich === true)
```

→ cờ dòng lệnh (`--enrich`) **thắng** file cấu hình — đúng thứ tự ưu tiên đã học ở P0. Nếu không phát hiện candidate nào (`discovered === 0`) thì bỏ qua.

4b. `enrichStaticEvidence` (:170-261) — chi tiết đầy đủ

- **Chọn công cụ để so sánh có kiểm soát** (:186): mặc định chỉ chạy 2 công cụ `functionSummary` + `pathConstraints` (đúng cặp mà judge tiêu thụ). `--static-tools` cho phép thêm `scanBuild/interproceduralFlow`. Vì sao mặc định hẹp: nếu mặc định chạy nhiều công cụ hơn thì mỗi lần thêm một công cụ mới vào hệ, số liệu baseline cũ đều phải đo lại — giữ mặc định hẹp giúp baseline **trùng từng byte** qua các bản code, chỉ mở rộng khi chủ động so sánh. (`callGraph` đã nổi dậy nhưng chưa dùng.)
- **scan-build** (:202-216): tùy chọn, chạy **1 lần cả dự án** (`scanBuildRun` → `scanBuildGetReport` → `attachScanBuildDiagnostics` gắn chẩn đoán vào mọi candidate khớp) — một "ý kiến tĩnh thứ hai" độc lập. Cần `buildCommand` vì scan-build hoạt động bằng cách chặn vào tiến trình build thật.
- **Vòng lặp theo bundle** (:218): `mapWithLimit(bundles, THRESHOLDS.discoveryConcurrency)` — số việc chạy đồng thời có trần, không mở không giới hạn. Vì sao cần trần: mỗi công việc là một lời gọi MCP thật đến analyzer — mở không giới hạn sẽ đập thẳng vào analyzer và gây timeout (câu chuyện 8→4 ở P4 §8).
 - **P0-1** (:223): đọc file từ `caches.files` — không đọc lại đĩa.
 - `functionSummary` (:227-237): gọi với `content` nạp sẵn (công cụ nhận nội dung file) → ghi vào store → **P0-2** `caches.tools.set('functionSummary', fsArgs, fs)` (khóa bỏ qua trường `content`).
 - `pathConstraints` (:239-247): tương tự, theo `lineNumber`.
 - `interproceduralFlow` (:249-260, tùy chọn của B2): **chỉ tăng recall** — chỉ thêm đường lộ khả dĩ, **không bao giờ dùng để trắng án**; chạy SAU `pathConstraints` để đường mới được nối thêm chứ không đè mất đường cũ. Cần **danh sách file toàn repo** (:189-196, `walkCFiles` giới hạn bởi `fileLimit` mặc định 2000) vì nó phải lần theo lời gọi hàm sang file khác — nếu chỉ tìm trong file của candidate thì recall bị chặn tại biên file.
- **Mọi lời gọi công cụ bọc trong `callToolRetryTimeout`** + bắt lỗi → chỉ `console.debug`: enrichment là kiểu "làm được càng nhiều càng tốt, hòng một phần không hủy cả buổi quét" — vì mục đích của nó là làm giàu bằng chứng, thiếu một phần bằng chứng vẫn được judge xử lý bình thường, còn nếu dừng cả quét vì một công cụ hỏng thì mất trắng toàn bộ phần còn lại.

4c. `runDynamicStage` (:558-589) — cửa chặn cho công thức động

```
chạy khi: no_llm && dynamicMode !== OFF && deps.dynamicClient && buildCommand
&& !dynamicRanInDiscovery
```

→ `runDeterministicDynamicStage(dynamicClient, bundles, {repoPath, buildCommand, ...})` — công thức build→LSan **không một dòng LLM nào** (đóng góp số 3: cô lập tính không-lặp-lại vào riêng judge). Thiếu `buildCommand` → thông báo "no_llm dynamic needs a build command; skipped" (:583-588). Comment bảo mật :555: giai đoạn này **thực thi code** — cùng phạm vi giam giữ với `llm_assisted` (chi tiết ở P6).

5. Giai đoạn Investigation — `runInvestigation` (:516-549)

Cửa kép: `deps.investigation && analysisMode === LLM_ASSISTED` — **no_llm không gọi hàm này** (vì giai đoạn này là phần duy nhất dùng LLM để thu thập bằng chứng; tắt nó đi là hệ hoàn toàn không phụ thuộc model). Bối cảnh truyền xuống (`investigation.ts:40-60`) có vài điểm tinh tế:

- **projectOwnershipNotes** (:46-49): các quy ước sở hữu bộ nhớ do allocator profiler phát hiện bằng LLM (ví dụ "cJSON_Add*ToObject chuyển quyền sở hữu cho cha"). Truyền cho LLM judge để phán quyết tôn trọng ngữ nghĩa riêng của dự án — loại tri thức mà quy tắc cứng không thể mã hóa hết được.
- **dynamicAlreadyRan** (:542): nếu discovery đã build + chạy rồi thì Stage B bỏ qua, giữ nguyên coverage đã gán — tránh build hai lần tốn thời gian.
- **InvestigationOutcome** (*investigation.ts:62-77*) — kết quả mà P3 sẽ phân tích sâu: **turns**, **agentDecisions[]**, **transcript** (ghi thành **transcript.json** để tái lập), **stepsLog**, **usage {inputTokens, outputTokens}**, **truncatedCalls** (số lần gọi LLM bị cắt ở trần token — tín hiệu chất lượng: một phán quyết hoặc lời gọi công cụ bị cắt nửa chừng có thể hỏng cấu trúc), và **staticContext: Record<bundleId, Record<tool, result>>** — bằng chứng theo từng bundle mà judge chấm điểm.
- **AgentMeta** (:16-20): **kind: 'main' | 'static' | 'dynamic' | 'harness'** — 4 loại agent, log tách theo từng agent để còn biết ai làm gì.
- **OrchestratorCommonDeps** (:26-38): 7 trường dùng chung giữa controller và investigation — chú ý **onUsageDelta** (harness đánh giá theo dõi chi phí trực tiếp cho trần chi phí từng case) và **requestPermission/getSteering/awaitResume** (cho phép TUI can thiệp giữa chừng).

6. Giai đoạn Judging — **runJudging** (:592-603)

```
for (const bundle of bundles) {
  if (bundle.verdict) continue; // LLM judge đã xử lý
  trong Stage A/D
  bundle.verdict = heuristicVerdict(bundle,
    investigationOutcome?.staticContext?.[bundle.bundleId] ?? {});
}
```

→ đây là **bộ chốt, không phải người phán chính**: mọi bundle chưa được ai kết luận sẽ được heuristic kết luận, với bằng chứng lấy từ **staticContext** (đúng chỗ mà enrichment/investigation đã ghi vào). Hàm dùng đúng **bundleId** làm khóa — nếu khóa lệch thì bằng chứng không đến được judge (liên hệ câu chuyện bundleId bị trùng ở §3).

7. Giai đoạn Reporting — **runReporting** (:606-644)

- Dựng **ScanMetadata** (:616-627): **scanId**, **workspacePath** + **sourceWorkspacePath**, **analysisMode**, **dynamicMode**, **fileLimit**, **buildCommand**, **startedAt** (lấy ở đầu **runScan**), **completedAt**, **status** COMPLETED.
- **reporter.buildReport(bundles, metadata, {agentDecisions})** → tóm tắt **confirmedLeaks/likelyLeaks** → phát COMPLETED (:637-641).
- Trả về **{report, bundles, investigation}** — **bản thân controller không ghi file; lỗi vào (headless/TUI) mới là nơi ghi** (writeReports/writeScanMetrics trong **domain/reportSink.ts** — đã thấy ở P1). Vì sao tách: controller giữ thuần logic, nơi ghi và tên file là chuyện của từng lỗi vào.

Tự kiểm P2

1. Hai lần chạy **no_llm** cho kết quả trùng từng byte — cơ chế nào đảm bảo dấu thời gian không phá điều đó? (*now có thể thay thế :648 — eval truyền đồng hồ cố định.*)
2. Vì sao enrichment và dynamic chạy song song không gây ghi đè? (*Ghi vào các trường khác nhau — staticEvidence so với evidence/dynamicCoverage.*)

3. `--static-tools scanBuild` ảnh hưởng gì đến baseline mặc định? (Không — công cụ phụ chỉ chạy khi được gọi tên; baseline 2 công cụ trùng từng byte.)
4. `interproceduralFlow` vì sao cần danh sách file toàn repo, và vì sao chỉ thêm đường mà không trắng án? (Phải lần theo lời gọi hàm qua file khác; mục đích là tăng recall — cấm đường ngược làm mất TP.)
5. 622 candidate → 65 bundle: lỗi gì, sửa gì? (bundleId cắt 6+10 ký tự mất dữ liệu phân biệt giữa phần đầu đường dẫn + phần cuối tên allocator; sửa bằng SHA1 toàn bộ khóa.)
6. Giai đoạn nào là chỗ duy nhất no_llm "nhìn thấy" phần động? (`runDynamicStage` — hoặc trong `discovery` nếu chạy `dynamic-only`; đều không qua LLM.)
7. `truncatedCalls` báo hiệu điều gì khi đọc `metrics.json`? (Số lần gọi LLM bị cắt ở trần token — phán quyết/lời gọi công cụ có thể hỏng cấu trúc; `freerdp_9fc23ad2` có 138.)

P3 — Stage A: sub-agent thu thập bằng chứng + chi phí token (1 buổi)

Mục tiêu

Trả lời 3 câu hỏi: (1) Stage A (các sub-agent LLM đi thu thập bằng chứng tĩnh) **chạy như thế nào trong code**, (2) bằng chứng thu được **đi về judge bằng đường nào** — và vì sao phần văn bản LLM tự viết không bao giờ tự quyết được, (3) **tiền token đổ vào đâu** — phân rã con số 44.3M token đo được. Học xong P3 bạn giải thích được cả cơ chế lẫn con số trước hội đồng.

File trung tâm: `apps/leak-inspector-tui/src/orchestrator/workflowInvestigation.ts` (nơi cài đặt `buildWorkflowInvestigationPhase` — giai đoạn mà `runScan` gọi ở P2 §5), kèm `domain/staticContext.ts`, `domain/subAgentPrompts.ts`.

1. Vị trí trong luồng — 4 stage nằm trong 1 giai đoạn investigation

Nhớ lại P2: `runInvestigation` chỉ chạy khi `analysisMode === LLM_ASSISTED`. `buildWorkflowInvestigationPhase` dựng **trình tự stage** trong giai đoạn này:

Stage A	<code>stageStaticEvidence</code>	— chia nhóm bundle, mỗi nhóm 1 sub-agent tĩnh
Stage B	<code>stageDynamicEvidence</code>	— bằng chứng động (công thức cố định HỎI worker LLM)
Stage B2	<code>targetedHarness</code>	— tùy chọn, cho bundle heuristic còn phân vân
Stage C	<code>synthesize</code>	— hợp nhất bằng chứng
Stage D	<code>judge</code>	— heuristic cho TẤT CẢ + LLM judge cho BORDERLINE + consensus tùy chọn

Toàn bộ stage chia sẻ một `WorkflowMutableState` (:124-135):

```
{ staticStore, dynStore, usage: {inputTokens, outputTokens},
  truncatedCalls, transcripts, decisions, stepLog, totalTurns }
```

→ **mọi con số trong `InvestigationOutcome`** (P2 §5) đều đến từ đúng object này: `usage` và tổng turns cộng dồn qua từng sub-agent, `transcripts` ghép lại thành `transcript.json`, `staticContext` chính là `staticStore`.

2. `runSubAgent` (:137-198) — khối xây chung của cả A/B/B2

Một hàm duy nhất chạy cả 3 loại sub-agent (`AgentMeta.kind: 'static' | 'dynamic' | 'harness'`):

- Chạy `queryLoop` (từ agent-core — P4) ở dạng **generator**: vòng `while(true)` gọi `gen.next()` — mỗi sự kiện (`tool_use`, `thinking`, ...) được (a) `bridge.handle` (gắn `AgentMeta`), (b) đẩy lên `onAgentEvent` để TUI vẽ dòng thời gian (► `calling_mcp ... / @ thinking...:177-191`), (c) `stepLog.record` (ghi log markdown).
- Khi kết thúc: cộng `usage` + `truncatedCalls` + `totalTurns` vào state, ghép messages vào `transcripts` (:193-197), bắn `onUsageDelta` (harness đánh giá theo dõi chi phí trực tiếp).
- **maxTurns** lấy từ cấu hình (`cfg.maxTurns`; Stage B được +10 :311) và **terminalTools** (gọi `DONE_*` là kết thúc).

3. Stage A — `stageStaticEvidence` (:200-247): chia nhóm + chuỗi bọc công cụ

Chia nhóm: bundles được chia thành các nhóm (`groups: LeakBundle[][]`), mỗi nhóm chạy **một sub-agent riêng** qua `mapWithLimit(groups, cfg.workflow.staticConcurrency)` — số nhóm chạy đồng thời có trần từ cấu hình.

Chuỗi bọc công cụ — trái tim của Stage A (:221-226):

```
withHostContent(                                ← nạp nội dung file thật từ đĩa (P0-1 file
cache)
  withStaticContextCapture(                      ← GHI kết quả vào staticStore (§4)
    withToolResultDedup(t, caches.tools),        ← P0-2: trùng cache thì không gọi
    lại MCP
    , staticStore, group),
    repoPath, caches?.files)
```

Ba lớp bọc này giải thích vì sao **LLM không thể "che giấu" hay "bịa" bằng chứng**: mỗi lần LLM gọi công cụ, kết quả (kể cả khi trùng cache) đều được chép cố định vào store; phần văn bản LLM tự viết không được ghi vào đâu cả. Đọc theo thứ tự thực thi: `host` → `capture` → `dedup` → `MCP` (comment :215-220 nói rõ thứ tự này và vì sao cache hit vẫn phải đi qua capture).

Bộ công cụ của agent (:227-231): các công cụ tĩnh nhận content + `read_file` + `buildDoneTool(DONE_STATIC)`. Prompt: `staticSubAgentSystemPrompt` + `staticSubAgentUserMessage(group)` (`subAgentPrompts.ts:50-74` — yêu cầu chạy đủ công cụ cho **từng** candidate, cho phép gọi nhiều công cụ song song trong một lượt, rồi gọi `DONE_STATIC`; **cảnh báo lịch sử dòng 47-49**: từng thử cho model tự do hơn và kết quả là mất độ chính xác mà không tiết kiệm được gì — đừng sửa nếu không chạy lại thí nghiệm so sánh).

Completion guard (:239-244) — cơ chế chống "làm biếng": khi agent gọi `DONE` mà vẫn còn bundle **chưa có mục trong staticStore**, guard trả về một lời nhắc buộc nó quay lại, chỉ đích danh các bundleId còn thiếu. Vì sao cần: một LLM có xu hướng kết thúc sớm; không có guard thì bằng chứng thiếu âm thầm và judge chấm trên dữ liệu hụt.

4. `foldStaticResult` (`staticContext.ts:59-174`) — 5 trường hợp ghi

Hàm áp kết quả công cụ vào store + bundle, khớp theo **file** (so tên cuối, dung sai với lệch đường dẫn tuyệt đối/tương đối) rồi tới **function/line**:

Công cụ	Ghi cái gì
<code>functionSummary</code> (:73-112)	<code>allocations/frees/hasExplicitFree</code> , <code>leaky_exit_paths</code> , <code>allocFreePairs</code> → <code>mergeStaticEvidence</code> ; thêm signature (returnType, parameters, isStaticLinkage) — phục vụ B2 viết harness đúng kiểu
<code>pathConstraints</code> (:113-130)	<code>feasiblePaths</code> , <code>constraints</code> , <code>earlyReturnCount</code> , <code>feasibleLeakPaths</code>
<code>ownershipSummary</code> (:131-145)	<code>ownershipSummary</code> + <code>ownershipType</code>
<code>astScan</code> (:146-157)	<code>earlyReturnCount</code> , <code>leakyExitPaths</code> theo function
<code>ownershipConventions</code> (:158-170)	thô : quy tắc rò rỉ nhắc tên function → đánh dấu <code>malloc_without_free</code>

Mỗi trường hợp đều **ghi kép**: vào `store` (bối cảnh theo bundle, judge đọc) và qua `mergeStaticEvidence(b, ...)` (vào chính bundle — nơi báo cáo sẽ ghi ra). Judge ở P2 §6 đọc đúng `staticContext[bundleId]` từ store này — vòng P2→P3 khép kín.

5. Stage B/B2/D — chỉ cần thấy đường (chi tiết ở P6/P7)

- **Stage B** (`stageDynamicEvidence` :249-320): 3 nhánh — (i) công thức cố định khi có `buildCommand` (`buildTarget`→`lsanRun`, **không LLM**, :275-286); (ii) công thức hổng → fallback sang worker LLM **chỉ khi tool_selector bật** (:287-296); (iii) worker LLM: các công cụ sanitizer bọc `withDynamicEvidenceCapture` — kết quả được ghi **cố định vào dynStore**, comment :298-301 là lời khẳng định kiến trúc: "the LLM only drives build/ run; it can no longer add or omit evidence that changes a verdict" (LLM chỉ lái build/chạy; nó không thể thêm hay bỏ bằng chứng làm đổi phán quyết). Completion guard đối xứng Stage A (:315-318: chưa có lần chạy sanitizer nào thành công thì cấm DONE).
- **Stage B2** (:322+): tùy chọn `workflow.targetedHarness.enabled` — worker LLM gọi `buildHarness/lsanRun/asanRun` trực tiếp cho bundle còn phân vân; việc nâng cấp lên fuzz tier sau đó là **code điều phối**, không phải một lượt LLM thêm.
- **Stage D**: LLM judge (`domain/llmJudge.ts`) + consensus (`packages/common/analysis/consensus-judge.ts`): `ConsensusConfig` {n, rule: majority/weighted/unanimous-to-flag, temperature, earlyStop}, `ConsensusVerdict` mang `samples []` + `agreement` + `evidenceFusion` {static: leak/clean/ambiguous, dynamic: confirmed/cleared/none} + `overridden` (heuristic phủ quyết một FLAG của consensus). Chi tiết ở P7.

6. Kinh tế token — phân rõ số đo được (MemHint, 3 lần chạy)

Nguồn: `results/memhint-llm-assisted-2026-08-28-run{1,2,3}/cases/*.json` (các trường `inputTokens/outputTokens/mcpCalls` theo từng case).

Khoản	Giá trị	Ghi chú
Tổng 3 lần chạy	≈44.3M (vào 26.5M / ra 17.8M)	llm_assisted x3; no_llm = 0

Khoản	Giá trị	Ghi chú
Mỗi lần chạy	vào 8.7–9.1M / ra 5.9–6.1M ≈ 14.7–15.2M	19 case mỗi lần
MCP calls	88.9K/lần ≈ 4.7K/case	công việc "đúng nghĩa" chỉ cần ~10–20 calls/case
Cá biệt freerdp_9fc23ad2	853K vào / 566K ra / 8,174 calls / 138 lần bị cắt	1 case ≈ 6% tổng 3 lần chạy
Theo dự án mỗi case	freerdp ~1.42M, openssl ~1.07M, vim ~0.66M, tmux/curl/redis ~0.45–0.6M	repo to → bối cảnh dày
LLM judge	2 phán quyết/3 lần chạy, khoảng vài chục K token	{ heuristic:17154, llm:2 } — không phải thủ phạm

Vì sao đắt — 3 cơ chế (nói về code):

- Vòng lặp agent tốn theo bình phương:** mỗi lượt gửi lại TOÀN BỘ hội thoại (system prompt + danh sách công cụ + mọi kết quả trước đó). 4.7K calls/case nhân với bối cảnh dồn dập = phần vào phình to. Guard ở §3 và lời nhắc completion cũng góp phần kéo dài vòng đời agent.
- Phần ra gồm cả suy luận:** deepseek-v4-flash là model có reasoning — 5.9–6.1M token ra mỗi lần chạy chủ yếu là suy luận + tham số lời gọi công cụ (metrics hiện chưa tách riêng reasoning_tokens — muốn chứng minh thì phải log thêm).
- Cache chỉ chặn phần trùng lặp:** P0–2 giúp các câu hỏi lặp lại, nhưng agent **mở rộng** câu hỏi liên tục (đọc file mới, công cụ mới) → cache không chặn việc mở rộng, chỉ chặn trùng lặp.

Kết cục đo được (FREEZE hàng 10): trên MemHint llm_assisted ≡ no_llm (TP12/FP0/FN14, F1 0.632±0.000) — 44.3M token **không mua được điểm F1 nào**, vì hybrid judge: heuristic xử lý ~17.1K phán quyết mỗi lần chạy, corpus này không có bundle nào borderline đủ để LLM judge lật phán quyết. Đây là bằng chứng trực tiếp cho khuyến nghị "cồng chọn lọc cho Stage A" — chỉ mở Stage A cho bundle mà bằng chứng cố định để lại mơ hồ (P9 sẽ ôn như hưởng phát triển).

Tự kiểm P3

- Sub-agent LLM có thể "khai bậy" bằng chứng không? Chỉ ra 2 lớp chặn trong code. (Không: **withStaticContextCapture** ghi mọi kết quả công cụ cố định vào store; Stage B dùng **withDynamicEvidenceCapture** — "the wrapper is the sole source". Văn bản LLM không ghi vào đâu.)
- Agent gọi DONE_STATIC khi còn bundle thiếu bằng chứng thì sao? (checkCompletion :239-244 trả về lời nhắc, chỉ đích danh bundleId — agent bị kéo lại làm.)
- Chuỗi bọc công cụ: thứ tự và tác dụng từng lớp? (host→capture→dedup→MCP; dedup nằm TRONG capture để cả cache hit vẫn được ghi vào store.)
- Vì sao Stage B đã có công thức cố định mà vẫn cần worker LLM dự phòng? (Dự án không khai buildCommand → công thức không chạy được; chỉ khi tool_selector bật mới fallback.)
- ConsensusVerdict.overridden** nghĩa là gì? (Heuristic phủ quyết một FLAG của consensus — lớp rẻ chống FP có quyền đè lớp đắt.)
- Phân rã 44.3M: judge chiếm bao nhiêu, Stage A chiếm bao nhiêu, 3 cơ chế nào làm Stage A đắt? (Judge gần như không đáng kể; Stage A chiếm gần hết; vòng lặp tốn bình phương + token suy luận phần ra + mở rộng câu hỏi liên tục.)
- Muốn giảm chi phí mà không đổi kiến trúc, cần cái nào an toàn nhất theo code hiện có? (P0–2 đã có sẵn; thêm trần lượt gọi/trần công cụ + chế độ đầu ra gọn cho công cụ — không đựng judge; đổi

model/phạm vi thì phải đo lại.)

P4 — packages/agent-core: vòng lặp gọi công cụ, callModel, compaction (1–2 buổi)

Mục tiêu

Đây là **bộ máy agentic tự viết, không dùng framework** (không LangChain hay thư viện ReAct — đóng góp "framework-free native tool-calling"). Học xong P4 bạn giải thích được: một lượt của agent đi qua những bước nào, lời gọi model được stream/thử lại thế nào, quyền hạn và thời gian chờ xử lý ra sao, và — phát hiện thú vị — **compaction trong hệ là cắt tỉa cố định, KHÔNG phải tóm tắt bằng LLM**.

Bản đồ package `packages/agent-core/src/`:

File	Nội dung
<code>loop.ts</code>	<code>queryLoop</code> — generator của vòng lặp gọi công cụ (trái tim)
<code>deps.ts</code>	Điểm tiêm phụ thuộc <code>CallModel/AgentDeps</code> — vòng lặp test được mà không cần mạng
<code>tool.ts</code>	Trừu tượng <code>Tool</code> + <code>buildTool</code> (mặc định an toàn) + <code>truncateResult</code>
<code>compaction.ts</code>	<code>pruneStaleToolResults</code> + <code>estimateTokens</code>
<code>providers/index.ts</code>	<code>buildCallModel</code> — bộ chia đường
<code>providers/openaiChat.ts</code> / <code>anthropic.ts</code>	2 backend (chat-completions dùng chung cho local+openai)
<code>providers/transport.ts</code>	<code>streamWithRetry</code> — stream + thử lại theo backoff + thời gian chờ
<code>mcp/mcpClient.ts</code>	MCP client HTTP (Streamable HTTP)
<code>mcp/mcpToolAdapter.ts</code>	<code>wrapMcpTool/loadMcpTools</code> — công cụ từ xa → Tool cục bộ

1. Điểm tiêm phụ thuộc (`deps.ts`) — vì sao vòng lặp test được

`CallModel = (req: CallModelRequest) => Promise<NormalizedResponse>` — vòng lặp KHÔNG biết nhà cung cấp nào; khi test, người ta tiêm một `callModel` giả → vòng agent chạy cố định mà không có mạng. `AgentDeps` gồm `callModel`, `uuid`, `now`, `log` (`productionDeps` :30–38). Vì vậy các test về steering/terminal-tool chạy không cần LLM thật — và quan trọng hơn, mọi hành vi của vòng lặp được kiểm chứng **tách rời** khỏi sự bất định của model.

`CallModelRequest` (:10–19): `systemPrompt` + `messages` + `tools` + `signal` + **`temperature?` riêng từng lần gọi (judge cố định 0 — có ghi chú trong code)**

- `onFirstChunk` (tín hiệu "đang nhận" cho giao diện).

2. `queryLoop` (`loop.ts`) — vòng đời một lượt

Generator phát `AgentEvent` cho nơi gọi (`runSubAgent` ở P3 §2) tiêu thụ. Một lượt đi qua đúng các bước:

1. **Trần lượt:** `turn < maxTurns` — hết ngân sách → kết thúc kèm lý do.
2. **Pause/Resume** (:127-132): khi `awaitResume` có sẵn — phát `{type: 'paused'}` và **chờ quyết định** `resume|abort`; `abort` → kết thúc với reason 'aborted'. Lướt thử lại **tái dùng số lượt — không trừ ngân sách** (comment :126).
3. **Gọi model** → `resp` (text + toolUses + usage + stopReason).
4. **Lời nhắc hoàn thành** (:170-176): gọi `checkCompletion()` (do từng stage cài — P3 §3/§5) — nếu còn việc dở → đẩy một tin nhắn nhắc + tăng `stopNudges` (có trần `maxStopNudges`) — guard có trần nên không thành vòng lặp vô hạn.
5. **Giải quyền hạn** (:196-220) — **tuần tự** vì có thể phải dừng chờ người phê duyệt: `deny` → quyết ngay; `ask` → gọi `requestPermission` từ ToolCtx (TUI hiện hộp y/n; headless mặc định 'allow'); công cụ không tìm thấy vẫn `allow` để **thực thi → trả lỗi "unknown tool"** cho model tự sửa.
6. **Chia luồng thực thi** (:244-255): chia toolUses thành **nhóm chạy song song** (`isReadOnly && isConcurrencySafe`) chạy `mapWithLimit(..., concurrency)` và **phần chạy tuần tự** lần lượt — mọi công cụ nặng của hệ dựa vào sự phân chia này để không đụng nhau.
7. **Kết quả theo đúng thứ tự yêu cầu** (:257-264): mỗi kết quả phát sự kiện `tool_result` rồi **gộp tất cả thành MỘT tin nhắn user** đẩy lại vào messages — đúng định dạng bắt cặp `tool_use ↔ tool_result` mà API yêu cầu.
8. **Chốt bằng terminal** (:266-271): công cụ tên trong `terminalTools` chạy **không lỗi** → kết thúc vòng lặp. (Chú ý: công cụ terminal chạy LỖI thì vòng lặp TIẾP TỤC — lỗi được trả về cho model để thử lại.)

3. Trừu tượng **Tool** (`tool.ts`) — mặc định là an toàn

- **Tool** (:33-48): tên, mô tả, schema (nội bộ: `zod inputSchema`; MCP: `inputJSONSchema` **chuyển nguyên văn, không chuyển đổi**), các hàm `isReadOnly/isConcurrencySafe` (quyết định phân chia ở §2.6), cổng quyền hạn, `call`, `mapResultToBlock`, `maxResultSizeChars`.
- **buildTool** (:55-68) điền mặc định an toàn: **giả định là công cụ ghi dữ liệu + không chạy song song được** → một khai báo chỉ cần nói cái KHÁC với chuẩn an toàn. Quên khai = công cụ chạy tuần tự = an toàn. Vì sao chọn fail-closed: sai lầm an toàn hơn là sai lầm nhanh.
- `DEFAULT_MAX_RESULT_CHARS = 100_000`; `truncateResult` (:75-79) cắt đuôi kèm dấu ... `[truncated N of M chars]` — giống cách MCP cắt.

4. Providers — stream + thử lại + 2 bẫy lịch sử

Bộ chia đường (`providers/index.ts:23-41`): `anthropic` → Messages API; mọi thứ khác (`local`, `openai`, profile `openai-compatible`) → chat-completions (`callOpenAiChat` — "local gateway và OpenAI API chỉ khác baseUrl/auth/model"). Sau mỗi lần gọi: `stopReason === 'max_tokens'` → phát `onTruncation` (:38) — đây chính là nguồn của `truncatedCalls` đã học ở P2/P3.

`callOpenAiChat` (`openaiChat.ts:29-90`) — các chi tiết đáng nhớ:

- Luôn `stream: true` + `stream_options.include_usage` (:40-41); bộ ghép stream **đặt lại mỗi lần thử lại** (:74-77) để không dính dữ liệu của lần cũ.
- Thời gian chờ 3 tầng từ cấu hình: `connectTimeoutMs` / `idleTimeoutMs` / `maxTotalMs` + `retries` (:66-70).
- **Bẫy 1 — mentionsJson** (:19-27): API OpenAI-compatible trả 400 nếu đặt `response_format: {type: 'json_object'}` mà prompt không có chữ "json" — hệ KIỂM TRA chứ không giả định; chỉ bật chế độ JSON khi `tools: []` (gọi một phát: judge/strategist/allocator-profiler) và prompt có nhắc json — tuyệt đối không đụng vòng lặp agentic.
- **Bẫy 2** (:86-88): bị cắt → `onNotice "...output may be incomplete"`.

- Cổng `jsonMode` (:48-56): các prompt không có công cụ đã tự ghi "JSON ONLY" — `response_format` chỉ là buộc cú pháp đúng, không thay thế lời dặn.

5. MCP client (`mcpClient.ts`) — giao thông + thử lại + theo dõi

- **Giao thông:** `@modelcontextprotocol/sdk Client + StreamableHTTPClientTransport` (:149-151); header `x-scan-id` gắn `correlationId` (:150) — nhờ đó phần request-context bên analyzer (`packages/common/src/mcp/request-context.ts`: `AsyncLocalStorage` + `pino` mixin) ghi log mọi dòng theo `requestId/correlationId`/công cụ mà không phải truyền tham số xuyên suốt chuỗi gọi.
- **connectOnce** nhớ kết nối (:145-159) — và **không bao giờ nhớ kết nối bị từ chối** (:154 "would poison every later call" — nếu nhớ một thất bại thì mọi lần gọi sau đều hỏng oan).
- **withRetry** (:133-142): thử lại có giới hạn + trễ ngẫu nhiên nhỏ; lỗi TẠM THỜI → `close()` client chết → lần sau nối lại sạch; abort/lỗi công cụ thì trả về ngay (việc thử lại dành cho giao thông, không che lỗi công cụ — vì lỗi công cụ thường là lỗi logic, thử lại cũng vậy).
- **callTool** (:174-209): đếm `callCount` (:179 — chỉ số hiệu quả #MCP calls/scan, chính là con số 88.9K ở P3!); thời gian chờ mặc định 60s; ưu tiên `structuredContent`, nếu không có thì phân tích text thành JSON (:195-204); `res.isError` → throw kèm gộp nội dung lỗi (:186-194).
- `ping()` = listTools thành công (:211-218) — preflight ở P1 dùng đúng cái này.

6. Chính sách chờ — ai quyết công cụ nào chạy song song

`apps/leak-inspector-tui/src/domain/mcpToolPlan.ts`:

- **CONCURRENCY_SAFE** (:15-22): mọi công cụ tính **trừ scanBuildRun**, + 4 công cụ valgrind đọc báo cáo + listRuns → `{readOnly, concurrencySafe, maxResultChars: 6000, timeoutMs: 30s}`.
- **SERIAL_HEAVY** (:25-28): `scanBuildRun` + mọi công cụ build/chạy sanitizer → `{concurrencySafe: false, **ask: true**, timeoutMs: **300s**}` — build có thể kéo dài nhiều phút; `ask` = cần phê duyệt (TUI) hoặc chính sách headless.
- **MAX_RESULT_CHARS = 6000** (:30-32): kết quả analyzer (bảng AST, đồ thị luồng) rất dài — trần thấp để bối cảnh nhỏ đủ cho model local. Đây là 1 trong 3 lý do bối cảnh không phình không kiểm soát (2 phần còn lại: compaction §7 và P0-2 dedup).
- **MCP_TOOL_PHASE** (:48-71): bảng tên công cụ → `ScanPhase` — dòng thời gian TUI tô đúng giai đoạn nhờ đây.
- Công cụ lạ → **mặc định tuân tự an toàn** (:43-44).

7. Compaction (`compaction.ts`) — hóa ra KHÔNG phải tóm tắt bằng LLM

Phát hiện đáng nhớ nhất của P4. Vấn đề (`comment` :1-11): vòng lặp nối thêm mọi kết quả công cụ và **gửi lại toàn bộ transcript mỗi lượt** → phiên dài phình vô hạn. Giải pháp được chọn: **cắt tỉa cố định, không gọi thêm LLM**:

- `pruneStaleToolResults(messages, keepRecentTurns)` (:47-70): tìm các tin nhắn user mang `tool_result` (mỗi lượt một tin), giữ nguyên `keepRecentTurns` lượt gần nhất; kết quả cũ hơn (từ 200 ký tự trở lên, chưa phải placeholder) được **thay nội dung bằng placeholder ngắn** `[elided: N chars ...]` — **thu nhỏ tại chỗ, không xóa khối** → cặp `tool_use` ↔ `tool_result` **không bao giờ vỡ** (API sẽ trả 400 nếu cặp vỡ).
- **Không đụng tới:** system prompt, tin nhắn user đầu tiên, văn bản assistant, phán quyết đã ghi.
- `estimateTokens` (:24-38): ước lượng ~4 ký tự/token khi gateway không báo usage.
- Ba lớp quản bối cảnh vây nhau: **cắt lúc sinh** (`maxResultChars=6000`) → **không sinh lại** (P0-2 dedup) → **cắt tỉa khi già** (prune theo lượt).

Nói về P3: compaction giải phóng bối cảnh nhưng phần chi phí bình phương của vòng lặp vẫn còn ở các lượt chưa bị cắt tia — 138 lần bị cắt của freerdp là bằng chứng vòng lặp chạy dài sát biên.

8. Bổ sung — thresholds.ts (câu chuyện con số)

`makeThresholds()` (:21-50): `borderlineLow/High = 0.35/0.7` (dải độ tin của heuristic được coi là `borderline` → xứng đáng có ý kiến LLM thứ hai — P7 sẽ dùng), cửa sổ code cho judge 6/5 dòng, `maxAllocFreePairsShown=12`, `maxFeasibleLeakPathsShown=5`, và **discoveryConcurrency** hạ từ **8 xuống 4**: khi để 8, số case chạy đồng thời (tối đa 6 trong `no_llm`) nhân 8 = **48 lời gọi MCP đồng thời** đập vào `static-analyzer` → **9/19 case MemHint bị timeout** (đã tái hiện trực tiếp). Cách sửa: giới hạn ở phía NGƯỜI GỌI; phía MÁY CHỦ sửa bằng `pool worker-thread` cho việc `parse`. Đây là câu chuyện tròn vẹn về tạo áp lực tải hai đầu — câu trả lời mẫu cho câu "hệ gặp nghẽn thật thì xử lý thế nào".

Tự kiểm P4

- 1. Vòng đời một lượt của `queryLoop`? (*trần lượt → gọi model → kiểm tra hoàn thành → giải quyền tuần tự → chia song song/tuần tự → gộp kết quả một tin nhắn theo thứ tự → kiểm tra terminal.*)
- 2. Vì sao giải quyền phải tuần tự? (*Có thể phải dừng chờ người phê duyệt trong TUI.*)
- 3. Công cụ terminal chạy LỖI thì vòng lặp dừng hay tiếp? (*Tiếp — lỗi trả về model thử lại; chỉ terminal thành công mới chốt.*)
- 4. `mentionsJson` chặn bẫy nào? (*API 400 khi response_format json_object mà prompt không nhắc json — kiểm tra rồi mới đặt, và chỉ khi không có công cụ.*)
- 5. Compaction trong hệ là gì, vì sao chọn cắt tia thay vì tóm tắt? (*Cắt tia cố định bằng placeholder — không tốn lần gọi LLM, không vỡ cặp tool_result, không phá tính lặp lại; tóm tắt bằng LLM vừa đắt vừa không lặp lại.*)
- 6. `callCount` trên `McpClient` đo gì, con số đó xuất hiện ở đâu trong phân tích chi phí? (*Số lần gọi MCP mỗi quét — 88.9K/lần chạy ở P3 §6.*)
- 7. Ba lớp quản bối cảnh của hệ? (*maxResultChars cắt lúc sinh; P0-2 tránh sinh lại; prune khi già.*)
- 8. Câu chuyện `discoveryConcurrency` 8→4 chứng minh điều gì? (*Nghẽn thật: 48 lời gọi đồng thời → 9/19 case timeout; sửa ở cả hai phía — giới hạn người gọi + pool worker máy chủ.*)

P5 — Bên trong static-analyzer: CParserService + 11 công cụ (1 buổi)

Mục tiêu

Đây là **bộ phận phân tích tĩnh** — và `CParserService` là nút thắt mà mọi công cụ đi qua. Học xong P5 bạn giải thích được: vì sao cache 2 tầng biến 54 giây thành 3 giây, vì sao container từng bị OOM dù chỉ chạy 5 worker, schema công cụ khai ở đâu (một nguồn duy nhất), và tầng dịch vụ xử lý lỗi parse thế nào để một file hỏng không hủy cả lần quét.

Bản đồ `apps/static-analyzer/src/`:

File	Nội dung
<code>main.ts</code>	NestJS DI context → chỉ phục vụ MCP/HTTP (:50061). Không REST/gRPC

File	Nội dung
mcp/static-mcp-server.ts	Đăng ký 11 công cụ từ catalog chung
services/c-parser.service.ts	Trái tim: tree-sitter + pool worker + cache 2 tầng
services/candidate-scan.service.ts	Discovery từ vựng + AST (P3 đã chạm)
services/ast-scan.service.ts	8 mẫu lỗi rò rỉ
services/ownership-analysis.service.ts	Ownership + quy ước
services/call-graph.service.ts / function-summary.service.ts / path-constraints.service.ts	Các phân tích còn lại
services/scan-build-adapter.service.ts	Bọc Clang scan-build
services/file-indexing.service.ts	indexFiles + bảo mật symlink

1. Bề mặt công cụ — 11 công cụ, một nguồn catalog duy nhất

`static-mcp-server.ts:43-127` — `createStaticMcpServer` đăng ký đủ 11 công cụ (`indexFiles`, `candidateScan`, `astScan`, `callGraph`, `functionSummary`, `interproceduralFlow`, `pathConstraints`, `ownershipSummary`, `ownershipConventions`, `scanBuildRun`, `scanBuildGetReport`) với:

- **Tên + mô tả + `inputSchema` (Zod) đến từ `STATIC_TOOL_DEFS`** trong `@cleak/common/mcp/tool-catalog` — "nguồn sự thật duy nhất" (:38-40). Vì sao một nguồn: client (agent-core) nhận `inputJSONSchema` nguyên văn từ chính catalog này → hai đầu không bao giờ lệch schema.
- **Mọi handler bọc `instrumentTool`** (:45-46): log started/finished/failed (tên, tham số đã che dữ liệu nhạy cảm, thời lượng, kết quả) — nhờ request-context đã học ở P4 §5.
- Handler chỉ là một dòng ủy quyền sang phương thức dịch vụ + bọc `ok()`.
- **Comment cũ đáng biết** (:21-23): file còn nhắc "the gRPC controller... both transports are behavior-identical" — gRPC **đã bị xóa** khỏi master; comment giữ lại như dấu vết lịch sử. Đừng để nó gây nhầm khi đọc.

2. CParserService — nút thắt + pool worker + cache 2 tầng

2a. Pool worker Piscina và câu chuyện OOM (:87-126)

- `STATIC_PARSER_WORKERS` (mặc định `số CPU - 1`), file worker riêng `workers/parse.worker.js` — **không tìm thấy → fallback parse trong tiến trình chính** kèm cảnh báo (build dev), không chết.
- **Mỗi worker giữ một tree-sitter parser dùng suốt đời** → bộ nhớ tăng **CỘNG DỒN theo thời gian sống, không theo từng tác vụ** — đây là lý do thật khiến 5 workers làm container bị OOM trong load test (không phải do số luồng). Cách sửa: `resourceLimits.maxOldGenerationSizeMb` (mặc định 512, chỉnh qua `STATIC_PARSER_MAX_OLD_GEN_MB`) — trần V8 **cho từng worker**, Piscina **tự dựng lại** worker chạm trần (khởi động lại rề, có kiểm soát) thay vì cả container chết. `idleTimeout` 5 phút để thu hồi worker rảnh — chặn luôn phân mảnh heap trong trường hợp xấu nhất. Comment nói thẳng: 512 là điểm khởi đầu thận trọng, **chưa đo lường** — cần xem `docker stats` rồi mới chỉnh.
- Nối về P4: đây chính là "phía MÁY CHỦ" của câu chuyện `discoveryConcurrency 8→4`.

2b. Cache 2 tầng — bộ nhớ LRU + đĩa (:143-240)

```

parse(content, filePath, ea, ed):
  key = sha1-base64url( "c|cpp" + content + ea đã sắp + ed đã sắp )      :150
  trúng bộ nhớ LRU? → trả về (cập nhật mới dùng nhất)                  :151-157
  trúng đĩa? → nạp vào bộ nhớ, trả về                                   :158-162
  trượt → runParse → ghi bộ nhớ + ghi đĩa                               :163-166

```

- **Khóa gồm cả extraAllocators/Deallocators** (đã lọc theo `^[A-Za-z_]\w*$` + **sắp xếp** — cùng profile dù gọi lệch thứ tự vẫn trùng khóa, và giữ được tính cố định) → profile khác = parse khác, đúng chủ ý.
- **base64url chứ không base64** (:148-149): khóa chính là tên file trong cache đĩa — base64 thường chứa `/` → vỡ đường dẫn.
- **Bộ nhớ LRU giới hạn theo byte** (:169-180): theo dõi `cacheTotalBytes`, đuổi mục cũ nhất khi vượt `CACHE_MAX_BYTES` (biến `STATIC_PARSER_CACHE_MAX_MB`, mặc định 256MB — sần 16MB).
- **Cache đĩa dưới `RUNS_DIR/ast-cache`** (:182-208): JSON theo từng khóa; khi ghi lỗi chỉ cảnh báo — **lỗi cache phải không bao giờ làm hỏng việc parse** (:205). Đọc lỗi → cảnh báo + tính lại.
- **Dọn dẹp giữ chỗ** (:214-240): chỉ giữ `STATIC_PARSER_DISK_CACHE_MAX_ENTRIES` (mặc định 5000) file mới nhất theo mtime, làm hết sức không hủy caller.
- **Kết quả đo được** (CLAUDE.md): quét lại dự án 143 file thật **54s/1.03GiB lạnh → 3s/266MiB ấm** — đây là kinh tế học của baseline sweep (cùng một checkout, đo lại nhiều cấu hình/lần chạy).

2c. Xử lý lỗi parse (:242-262)

Parse lỗi (worker hoặc trong tiến trình) → trả `{functions: [], functionNames: []}` — không bao giờ throw. Tầng trên xử lý êm: `candidateScan` cảnh báo + rơi về quét từ vựng; các dịch vụ khác trả kết quả rỗng. Một file hỏng không hủy quét — bất biến "một file xấu chỉ làm mất tối đa một candidate".

2d. C so với C++ (:132-141)

`isCppPath` → dùng `tree-sitter-cpp` thay vì `tree-sitter-c` — nếu không sẽ **parse sai**

`new/delete/template/::/range-for` (nối thẳng vào điểm yếu recall của family C++ `new/delete` trên Juliet toàn corpus).

3. Tầng dịch vụ — các điểm tinh đã soi

candidateScan (đọc đủ ở P0): các biểu thức gộp + từ chối nhanh 1 regex mỗi dòng; xác định hàm chứa từ **biên hàm AST** (LAMEd chấm theo hàm → gán sai hàm = mất flaw); **sanitizeSource** xóa comment/chuỗi **giữ nguyên số dòng** (block comment mất ký tự xuống dòng sẽ đẩy lệch toàn bộ dòng phía dưới — file Juliet mở đầu bằng header ~14 dòng!); **F3**: candidate tổng hợp cho lỗi tham-số-con-trò (tham số được free trên vài đường, mất trên đường khác) — chỉ chạy trong container (cần `tree-sitter`), trên host là không-op.

astScan — 8 mẫu: `early-return`, tích lũy trong vòng lặp, điều kiện, `strdup`, `realloc` dùng sai, thiếu kiểm tra NULL, trường struct không được giải phóng, phân tích đường thoát từ CFG + **computeConfidence** (critical/high → high; từ 3 mẫu trở lên → high...).

ownership-analysis: **summarize** parse **đồng thời qua pool nhưng ghép kết quả theo thứ tự file gốc** — chạy đồng thời mà kết quả cố định (:26-38). **inferOwnershipSummary** (:69-122) — phần "khớp MemHint/LAMEd": phân vai **allocator/deallocator/both/neither** + **ownershipCarrier** (**return_value**: quyền sở hữu chuyển cho caller / **parameter** ở vị trí i: quyền sở hữu bị tiêu từ caller / none) kèm lý do viết bằng tiếng người — chính là trường **OwnershipSummary** mà **foldStaticResult** (trường hợp

3) và LLM judge tiêu thụ. `conventions` sinh 4 loại quy tắc (`leak_risk`, `missing_free`, `loop_leak`, `early_return_leak`) — đúng 4 loại mà `foldStaticResult` (trường hợp 5) nhận diện.

4. `scan-build-adapter` — clang ngay trong container

- Biến môi trường: `RUNS_DIR` (mặc định `./runs`), `SCAN_BUILD_BIN` (mặc định `scan-build`) (:42-43); sandbox chuẩn `WORKSPACE_ROOT`, nếu không có thì `/workspace` (:52).
- `run(projectPath, buildCommand, timeoutSec) → {runId}`; chặn vào **build thật** của dự án; `getReport(runId) →` các findings (chẩn đoán clang đã chuyển thành cấu trúc). Chạy **trực tiếp trong container static-analyzer** (clang được cài sẵn trong image) — **không** lồng `docker run`, không mount `docker.sock` (khác thể hệ trước của hệ — vì lồng Docker vào Docker vừa chậm vừa mở thêm bề mặt rủi ro).
- Phía điều phối (P2 §4b): tùy chọn qua `--static-tools scanBuild`, chạy 1 lần mỗi dự án, `attachScanBuildDiagnostics` gắn chẩn đoán vào mọi candidate khớp — "ý kiến tĩnh thứ hai" cố định.

5. `indexFiles` — bảo mật symlink (:10-107)

- `realpathSync(rootPath)` làm **gốc chuẩn** (:22-24) — symlink gài trong repo không thể lái việc quét ra `/etc`, `~/ssh`.
- Duyệt bằng `lstat` (THẤY symlink thay vì đi xuyên); theo symlink **chỉ khi đích thật nằm trong gốc** (:76-88); bỏ qua thư mục ẩn/node_modules/ `pycache`; `fileLimit` mặc định **10000**.
- Mọi lỗi từng mục (không quyền, symlink gãy) → bỏ qua + ghi vào errors, không hủy việc duyệt.

6. Đối xứng với cache phía TUI (`fileContentCache.ts`)

`FileContentCache` (cache `files` của P2): khóa = **đường dẫn tuyệt đối đã resolve**; mục = `{stamp: "size:mtimeMs", content}`; **stamp được kiểm lại bằng 1 statSync rẻ mỗi lần đọc** — file đổi giữa chừng lần quét thì được đọc lại đúng đắn, file không đổi chỉ trả giá một lần stat chứ không bao giờ trả giá đọc nội dung lần hai. Phạm vi theo từng lần quét như `scanCaches.ts` (Map rỗng lúc tạo, hủy khi quét xong).

Tự kiểm P5

1. Vì sao container OOM dù chỉ 5 worker? (*Parser dùng suốt đời mỗi worker → bộ nhớ tích lũy; sửa bằng trần heap 512MB/worker + Piscina tự dựng lại, không phải giảm số luồng.*)
2. Khóa cache gồm những gì? Vì sao `base64url` và vì sao sắp xếp allocators? (*"c/cpp" + nội dung + ea + ed; khóa = tên file cache đĩa; sắp xếp để thứ tự gọi khác nhau vẫn trúng cache và giữ tính cố định.*)
3. 54s→3s đến từ tầng cache nào, vì sao quan trọng với đánh giá? (*Bộ nhớ LRU + đĩa; baseline sweep quét lại cùng checkout nhiều cấu hình/lần chạy — lần sau ăn cache đĩa.*)
4. Parse hỏng ở một file thì sao? (*Trả danh sách hàm rỗng — không throw; candidateScan rơi về quét từ vựng; bất biến một-file-xấu.*)
5. Việc giữ nguyên số dòng khi làm sạch nguồn bảo vệ điều gì? (*Neo số dòng của candidate/bằng chứng — block comment mất ký tự xuống dòng sẽ đẩy lệch cả file phía dưới.*)
6. `ownershipCarrier` cho biết gì, ai tiêu thụ? (*Quyền sở hữu rời hàm qua giá trị trả về/tham số bị tiêu/không có, kèm lý do — foldStaticResult trường hợp 3 + LLM judge + đối chiếu MemHint/LAMeD.*)
7. `scan-build` chạy ở đâu, vì sao không lồng Docker? (*Ngay trong container static-analyzer, clang có sẵn; không lồng Docker + không docker.sock = bề mặt rủi ro và độ phức tạp ít hơn.*)
8. Phần bảo mật của `indexFiles`? (*Gốc chuẩn qua `realpath` + `lstat` + theo symlink chỉ khi đích thật trong gốc.*)

P6 — dynamic-analyzer + mô hình tin cậy bảo mật (1 buổi)

Mục tiêu

Đây là bộ phận **duy nhất thực thi code không tin tưởng** (build + chạy chương trình của chính repo đang quét). Học xong P6 bạn trả lời được: công thức build → LSan chạy thế nào trên Linux và macOS, 4 trạng thái coverage trung thực là gì, bằng chứng tĩnh và động được nối với nhau và chấm độ tin thế nào, và — phần quan trọng nhất khi bảo vệ — **mô hình tin cậy chấp nhận rủi ro nào, chặn rủi ro nào, bằng cơ chế gì**.

Bản đồ `apps/dynamic-analyzer/src/services/`: `build-target.service.ts` (buildTarget), các dịch vụ valgrind (4 công cụ), `asan-run/lsan-run/run-binary.service.ts`, `harness-build.service.ts` (B2), `libfuzzer-run.service.ts`, `run-manager.service.ts` (lưu các lần chạy), `result-parser.service.ts` (chuẩn hóa kết quả), `path-guard.ts` (sandbox), `safe-exec.ts` (`runConfined`).
Phía điều phối đối xứng: `domain/dynamicEvidence.ts`.

1. Bề mặt công cụ — 11 công cụ, cùng mẫu với phía tĩnh

`dynamic-mcp-server.ts` đăng ký từ **cùng tool-catalog** (P5 §1) — `buildTarget`, `valgrindMemcheck`, `valgrindGetReport`, `valgrindListFindings`, `valgrindCompareRuns`, `asanRun`, `lsanRun`, `runBinary`, `buildHarness`, `libfuzzerRun`, `listRuns`. Chính sách cờ (P4 §6): các công cụ build/chạy thuộc nhóm `SERIAL_HEAVY` + `ask`; chỉ 4 công cụ đọc báo cáo/danh sách là `CONCURRENCY_SAFE`. Đồng thời gian TUI map vào `ScanPhase.DYNAMIC` qua `MCP_TOOL_PHASE`.

2. buildTarget — trái tim của công thức (`build-target.service.ts`)

2a. Giữ trong sandbox trước, build sau (:21-38)

`assertInsideWorkspace(projectPath)` **chạy trước mọi việc khác** — đường dẫn do người gọi cung cấp phải nằm trong sandbox (chi tiết ở §4) — rồi mới kiểm tra sự tồn tại. Vì sao kiểm trước: một đường dẫn rời sandbox phải bị chặn trước khi bất kỳ lệnh shell nào được thực hiện với nó.

2b. RESIDUAL RISK — comment cần thuộc (:57-64)

`buildCommand` là lệnh shell do người vận hành cung cấp và được **thực thi nguyên văn** qua `execSync` — điều này là bản chất của "build dự án đang quét" (Make/CMake/autotools đều cần shell). Hệ không tự nhúng nội dung không tin tưởng vào lệnh này; kiểm tra `WORKSPACE_ROOT` ghim thư mục làm việc về thư mục dự án. **Build script của repo không tin tưởng có thể chạy lệnh gì đó — đây là rủi ro ĐƯỢC CHẤP NHẬN của mô hình tin cậy** (giảm nhẹ bằng `--network none` / giới hạn bộ nhớ/tiến trình / giam giữ container), KHÔNG phải lỗi chèn lệnh từ code này.

Đây là kiểu "mô hình đe dọa trung thực" mà hội đồng đánh giá cao: phân biệt rõ **rủi ro chấp nhận** (build script không tin tưởng) với **điều không cho phép** (đường dẫn ra ngoài sandbox do người gọi cung cấp).

2c. Thích ứng macOS (:260-280 + :189-196)

- `adaptSanitizerFlags`: trên macOS, đổi `-fsanitize=leak` → `-fsanitize=address` (trên macOS LSan nằm trong ASan, không chạy riêng được); thay cả trong biến `CMAKE_C_FLAGS/CMAKE_CXX_FLAGS`.
- `shouldUseDockerBuild`: macOS + lệnh build có cờ sanitizer → build trong Docker Linux (`gcc:latest`; script `.mcpvul_docker_build.sh` được ghi vào dự án, nguồn mount được chuẩn hóa

bằng `realpathSync`, tham số `docker` truyền dạng mảng không qua shell :130-147 — đường dẫn không thể chèn lệnh; `--network none` mặc định (mở qua `DYNAMIC_BUILD_NETWORK` vì một số build thật cần tải gói), `--memory 1g`, `--pids-limit 512` — chống rò dữ liệu ra ngoài và chống lệnh sinh tiến trình tràn (fork-bomb).

- **Valgrind: chỉ chạy trên Linux** — không chạy trực tiếp trên macOS; luôn qua Docker khi cần (CLAUDE.md ghi rõ).

2d. `findBinary` — tin báo cáo build một nửa (:198-235)

Giá trị `-o <file>` được grep từ **đầu ra của build script không tin tưởng** → chỉ nhận khi: `resolve()` (gập các `..`) → `isPathInside(candidate, projectPath)` → tồn tại → chạy được (:202-207). Dự phòng: danh sách tên thường gặp (`a.out`, `build/app`, ...) → tìm đệ quy sâu 3 (bỏ `.git/node_modules`) → cuối cùng là `a.out`. **Phía điều phối cũng có dự phòng tương tự** (`dynamicEvidence.ts:271-273` — quy ước `a.out` khi discovery để trống).

3. Công thức và bằng chứng — phía điều phối (`domain/dynamicEvidence.ts`)

- **`runDeterministicDynamic`** (:242-282): công thức cố định — tìm đúng 2 công cụ `buildTarget` + `lsanRun` (:252-253), build → lấy `binaryPath` → `lsanRun` bọc `withDynamicEvidenceCapture`. Build hỏng/ném lỗi → thông báo + **trả false** → nơi gọi fallback sang worker LLM (chỉ khi `tool_selector` bật — P3 §5). Trả về `store.runs.some(r => r.success)`.
- **`reconcileDynamicEvidence`** (:216-223): gập findings của mọi lần chạy thành công vào bundles qua `attachToBest` — comment khẳng định: **làm nhiều lần cho cùng kết quả, không phụ thuộc thứ tự** — cùng store + bundles luôn cho cùng bằng chứng.
- **`computeDynamicCoverage`** (:284-290) — **4 trạng thái trung thực**: `dynamic_off` (không bật) / `not_exercised` (không có lần chạy thành công) / `exercised_clean` (chạy thành công, không có rò khớp) / `exercised_leak` (chạy thành công, có rò khớp). Comment trong `evidence.ts:12-17`: không suy từ `evidence.length` vì **"chạy sạch" ≠ "chưa từng chạy"** — sự trung thực này là tín hiệu mà công precision của judge cần.
- **`DynamicRunRecord.targeted`** (:44): bằng chứng từ harness B2 (nhắm đúng candidate) khác với lần chạy cả-chương-trình của Stage B — judge cân nặng khác nhau.

4. Nối tĩnh với động — chấm độ tin của mỗi nối

`evidence.ts:4-9` — `CorrelationMethod` 5 bậc: `file_line_exact` → `file_line_near` → `function_match` → `file_only` (yếu) → `none`. Trên `LeakEvidence` (:33-68) còn có: `leakKind` (phân loại Valgrind/ASan-LSan), `allocStack` (khung người dùng + thư viện, `isUserFrame`), `signature` (SHA1 tránh trùng, do bộ chuẩn hóa sinh), **`correlationConfidence` từ 0 đến 1** (phương pháp + khoảng cách dòng + cùng họ allocator — để phá thế cân khi một finding khớp nhiều candidate gần nhau), `correlatedToCandidate`, `correlationDistanceLines`. Đây là nguyên liệu của đóng góp số 4 ("correlationMethod phân biệt mỗi nối mạnh với mỗi nối yếu").

Bổ sung phía tĩnh (cùng file): `CrossFileFreedVia` (:134-143) + `FreedViaCaller` (:145-157) — cặp gương chống 2 nhóm dương giả của Juliet (flow-variant ≥21: hàm "sink" nằm file khác; 42-45/61-68: dữ liệu trả qua caller) — `callGraph` tính `freedCrossFile/freedViaCaller`. **`OwnershipSummary.verification`** (:89-96): khẳng định tĩnh (`ownershipCarrier`) được **kiểm chứng động bằng harness** (thử "gọi một lần, không free" dưới ASan) — `refuted` → hạ carrier về `'none'` ngay tại chỗ (`ownershipVerification.ts`) — ví dụ đẹp của "khẳng định tĩnh bị phần động kiểm lại".

5. B2 và tầng fuzz

- `buildHarness (harness-build.service.ts)`: biên dịch + nối một **harness nhằm từng candidate** với **cờ biên dịch thật của dự án** (lấy lại qua `compile_commands.json` — `CompileEntry`) thay vì build cả dự án; các binary ghi vào `RUNS_DIR` (thư mục sản phẩm — đó là lý do path-guard cho phép chạy từ đó, §6); dọn dẹp giữ các thư mục run cũ (cùng mẫu với dọn cache đĩa ở P5 §2b).
- `libfuzzerRun (libfuzzer-run.service.ts:27-60)`: ngân sách mặc định **15s** (`-max_total_time`, sần 1s), thời gian chờ = ngân sách+30; tham số `-runs=-1 -close_fd_mask=3`; biến môi trường `LSAN_OPTIONS=verbosity=1:log_threads=1,ASAN_OPTIONS=detect_leaks=1`; **`assertExecutablePath(binaryPath)` trước tiên** (:34-40); đầu ra stderr/stdout → `parseLsanOutput`. Chỉ đặt tới qua B2 (việc nâng cấp do code điều phối quyết — không phải một lượt LLM thêm).

6. path-guard.ts — viên ngọc bảo mật (đọc đủ 152 dòng)

Trước khi file này tồn tại, `WORKSPACE_ROOT` chỉ nằm trên giấy (trong SECURITY.md) mà không được ép buộc — người gọi có thể đưa đường dẫn bất kỳ (:1-9). Bây giờ:

- **Chọn gốc** (:51-68): biến `WORKSPACE_ROOT` → nếu không có: `/workspace` nếu tồn tại (docker-compose mount repo vào đó), còn không thì `process.cwd()` — và **cảnh báo LÊN MỘT LẦN** khi dùng mặc định để người vận hành phải ghim rõ ràng.
- **`assertInsideRoot`** (:97-120) — 4 lớp:
 1. `resolve()` gập các `../`;
 2. so tiền tố **tính cả dấu phân cách** (`rootReal + sep`) — đường lân cận như `/root2/x` KHÔNG qua được kiểm tra của `/root`;
 3. **`realpath` tổ tiên tồn tại sâu nhất** (:80-91): thành phần chưa tồn tại không thể là symlink; mọi tổ tiên tồn tại được chuẩn hóa → symlink chảy ra ngoài bị từ chối;
 4. so với **`realpath(gốc)` — chính gốc đi qua symlink cũng đúng** (macOS `/var` → `/private/var`, `/tmp` → `/private/tmp`, bind-mount).
- **Phần đuôi chưa tồn tại được phép** (:109-111): `resolve()` đã gập `..`, và thành phần chưa tồn tại không thể là symlink — không có lối thoát.
- **`assertExecutablePath`** (:136-147): cho phép 2 gốc — `WORKSPACE_ROOT` hoặc `RUNS_DIR` (nơi `buildHarness` đặt binary **do máy chủ tự sinh**, mã run đã làm sạch — không mở lối cho người gọi kiểm soát nội dung).
- Mọi lần từ chối ghi log `PATH_REJECTED` tập trung tại đây (:114-116) — không rải rác ở 7 nơi gọi.

Tự kiểm P6

1. Mô hình tin cậy chấp nhận rủi ro gì, chặn gì? (*Chấp nhận: build script không tin tưởng có thể chạy lệnh — bản chất của việc build; chặn: đường dẫn ra ngoài sandbox, không mạng + giới hạn bộ nhớ/tiến trình, tham số docker dạng mảng, đường binary phải resolve + nằm trong phạm vi.*)
2. Trên macOS công thức chạy thế nào? (*Đổi `-fsanitize=leak` → `address`; build qua Docker Linux nếu lệnh build có cờ sanitizer; Valgrind chỉ Linux/Docker.*)
3. 4 trạng thái `DynamicCoverage` và vì sao gọi là trung thực? (*off/not_exercised/exercised_clean/exercised_leak — phân biệt "chạy sạch" với "chưa từng chạy", không suy từ số lượng bằng chứng.*)
4. `file_only` nghĩa là gì, vì sao vẫn giữ? (*Mối nối yếu — chỉ khớp theo file; vẫn gắn để judge thấy nhưng cân nặng thấp — phân biệt mạnh/yếu là đóng góp số 4.*)
5. `OwnershipSummary.verification = 'refuted'` xảy ra khi nào, hệ xử lý thế nào? (*Harness "gọi một lần, không free" dưới ASan bác khẳng định tính → hạ carrier về none ngay tại chỗ — phần tĩnh bị*

phần động kiểm lại.)

6. Vì sao `assertExecutablePath` cho `RUNS_DIR` mà không sợ lộn? (Sản phẩm do máy chủ tự sinh, mã run đã làm sạch — người gọi không kiểm soát nội dung.)
7. macOS `/tmp` → `/private/tmp` phá kiểu kiểm tra nào, sửa thế nào? (So tiền tố văn bản — sửa: so `realpath(gốc)` với `realpath(tổ tiên tồn tại sâu nhất)` — đúng cả khi gốc đi qua symlink.)

P7 — Tầng judge + mô hình dữ liệu @c leak/common (1 buổi)

Mục tiêu

Đây là **bộ phận ra phán quyết** — nơi mọi bằng chứng từ P2–P6 hội tụ thành verdict. Học xong P7 bạn giải thích được: bảng điểm heuristic (mỗi tín hiệu mấy điểm, hai cổng precision hoạt động ra sao), LLM judge nhận đầu vào gì và phân tích đầu ra thế nào, consensus gộp các phiếu với heuristic ra sao, và một verdict trở thành những file artifact nào.

Bản đồ:

File	Nội dung
<code>packages/common/src/analysis/heuristic-judge.ts</code>	Trái tim: <code>judgeHeuristically + enrichLeakVerdict</code>
<code>packages/common/src/analysis/consensus-judge.ts</code>	<code>judgeByConsensus</code> , <code>sampleWithEarlyStop</code> , <code>combineVerdicts</code>
<code>apps/leak-inspector-tui/src/domain/llmJudge.ts</code>	Prompt + phân tích verdict LLM (đơn + theo lô)
<code>apps/leak-inspector-tui/src/domain/judgeVerdictCache.ts</code>	Cache verdict theo băm prompt
<code>apps/leak-inspector-tui/src/domain/thresholds.ts</code>	Dài borderline (P4 §8 đã đọc)
<code>packages/common/src/analysis/reporting.ts + reporting/</code>	5 trình dựng báo cáo
<code>apps/leak-inspector-tui/src/domain/reportSink.ts</code>	Ghi các artifact

1. Heuristic judge — bảng điểm đầy đủ (`heuristic-judge.ts:46–351`)

Đầu vào: `bundle + staticContext[bundleId]` (P2 §6) + **đọc file nguồn thật** (`readFullFile`). Các ngưỡng **ĐÓNG BẮNG** `JUDGE_VERDICT_THRESHOLDS = {confirmed: 0.7, likely: 0.4}` (:34) — đánh giá không cho tinh chỉnh theo từng dự án (điều chỉnh có giới hạn chỉ dành cho vận hành thật) ⇒ kết quả đánh giá cố định.

Tín hiệu	Điểm	Ghi chú
Rò rỉ động khớp <code>definitely_lost/asan_leak/indirectly_lost</code>	+0.5	Lấy mức mạnh nhất, KHÔNG cộng dồn nhiều finding (:81-114)
Rò rỉ động khớp <code>possibly_lost</code>	+0.2	
Rò rỉ động khớp, loại không rõ	+0.4	
Finding không khớp candidate	0 (bỏ qua)	"Rò rỉ ở chỗ khác cùng file không phải bằng chứng cho lệnh cấp phát này" — từng là +0.15, là lỗi precision (:100-103)
Cấp phát <code>unpaired</code> của biến candidate (± 1 dòng)	+0.25	
Cấp phát <code>conditional</code>	+0.15	
Không có free nào trong hàm	+0.25	
Đường lộ khả dĩ	+0.2	
Chẩn đoán scan-build khớp	+0.15	Khớp theo biến bị rò (<code>pointed to by 'x'</code>) chứ không chỉ theo cửa sổ dòng — scan-build báo tại ĐIỂM RÒ (:150-161)
Lộ theo đường cụ thể (<code>conditional</code> + đường khả dĩ, hoặc biến nằm trong <code>unreconciled</code>)	+0.15	"Mẫu lỗi rò phổ biến nhất của dự án thật" (cJSON thiếu free một nhánh) — tín hiệu mạnh (:167-186)
Vai trò allocator + carrier <code>none</code>	+0.15	
Quyền sở hữu đã chuyển (trừ điểm)	-0.25 (-0.1 nếu có rò khớp)	Tín hiệu dương giả mạnh: trách nhiệm của caller — nhưng KHÔNG trắng án khi có lộ theo đường cụ thể ("đường MẤT đối tượng không phải đường chuyển quyền sở hữu" :205-208)
Quy ước <code>malloc_without_free</code>	+0.15	
Early return + có cấp phát	+0.1	
Candidate độ tin cao	+0.1	
Phân tích cấu trúc <code>high / medium</code>	+0.5 / +0.25	

2 đường rút ngắn LIKELY_FALSE_POSITIVE (độ tin 0.8) — cả hai đều **cấm nâng lên LLM** (vì LLM chỉ thấy đoạn code vài dòng quanh candidate, không thấy hàm sink):

- freedViaCallee** (:234-242): con trở được hàm callee (sink) free (Juliet good→goodSink) — "quyền sở hữu bị tiêu bởi sink". Trừ khi có rò rỉ động khớp.

2. **Gương đối xứng: `freedViaCaller`** (:244-274): hàm cấp phát và TRẢ VỀ con trỏ, caller giải phóng — 2 đường xác nhận (`se.freedViaCaller` theo toàn dự án, hoặc `structuralLikelihood === 'low'` trên INTERPROCEDURAL_LEAK). Comment kể nguyên vẹn case `char_calloc_42` (điểm 0.85 dương giả → xem lại transcript phát hiện).

`hasStrongSignal` (:305-310) = có rò rỉ động khớp || phân tích cấu trúc high || unpaired || lộ theo đường cụ thể || `malloc_without_free` — dùng cho 2 cổng:

- **Cổng 1** (:319-327): `dynamicallyCleared && !hasStrongSignal` → `likely_false_positive` — lần chạy sạch là bằng chứng tha thứ TRỪ khi có tín hiệu quyết đoán phản bác. **Comment dài :292-304 là bài học lớn**: trên dự án thật, `buildTarget->lsanRun` chạy chương trình KHÔNG tham số — `curl/vim/openssl` hầu như không chạm logic ứng dụng, nên `exercised_clean` ở đây nghĩa là "chưa chạy tới", không phải "đã chứng minh sạch" — cổng 1 cũ từng để một lần chạy sạch "mù" đè lên bằng chứng tĩnh tốt (cổng 2 hồng tương tự, đã sửa bằng việc kéo `hasStrongSignal` lên trước cả 2 cổng).
- **Cổng 2** (:329-339): muốn FLAG mà không có tín hiệu mạnh → hạ về `UNCERTAIN` — "một đồng tín hiệu từ vệt yếu không đủ để khẳng định rò rỉ".

Kết quả: độ tin = điểm đã chặn (`uncertain: max(clamped, 0.3)`), kèm `repair_suggestion`, `rootCause`, `repairDiff` từ `analyzeLeakHeuristically`.

2. `enrichLeakVerdict` (:358-388) — diff của LLM bị kiểm lại

Mọi verdict rò rỉ phải có `rootCause` + `repairDiff`. **Diff do LLM đưa chỉ được dùng khi: `isDiffApplicable`** (các dòng gốc khớp nguyên văn với file tại `startLine :391+`) $\wedge$ `diffAddsCleanup` $\wedge$ `diffIsMinimal` — sai → thay bằng diff cố định sinh từ chính nguồn. Phần giải thích của LLM (tool LLM/CONSENSUS) được giữ nguyên (:372-385). Đây là mẫu "LLM đề xuất, bộ kiểm tra quyết".

3. LLM judge (`llmJudge.ts`) — prompt, phân tích, cache

- **`SYSTEM_PROMPT`** (:51-56): bắt buộc trả JSON đúng khuôn `{verdict, confidence, explanation, evidence}` + `CALIBRATION_RULES` chia sẻ nguyên văn với biến thể theo lô (:58+).
- **`parseVerdict`** (:202-218): `JSON.parse` → hồng thì **dùng regex trích object `{...}` ra khỏi văn bản quanh** — tinh thần "ghi LÝ DO hồng thay vì im lặng rơi về heuristic". Zod `VerdictResponseSchema` (:164-169) + kiểm chuỗi verdict hợp lệ (:181) + chặn confidence vào [0,1] (mặc định 0.5).
- **Biến thể theo lô** (:220+): quét các object `{...}` cấp đầu, đếm ngoặc đúng sâu — chấm N bundle trong một lần gọi.
- **`judgeVerdictCache`**: cache verdict **khóa theo băm của prompt** — comment :48-50: sửa prompt = tự làm hồng khóa cache cũ, không ai phải nhớ tăng số phiên bản.

4. Consensus judge (`consensus-judge.ts`)

- **`judgeByConsensus`** (:381-395): lấy `sampleJudge` N mẫu ($n=1$ **có ý trùng với judge LLM đơn** — baseline so sánh miễn phí :377-379) → `deriveFusion(bundle)` → `combineVerdicts(samples, heuristic, fusion, cfg)` — **heuristic vẫn chạy bên trong consensus để gộp**.
- **`ConsensusConfig`** (P6 §5 đã liệt kê): `{n, rule: majority|weighted| unanimous-to-flag, temperature>0 để có tính đa dạng thật, concurrency, earlyStop}`.
- **`sampleWithEarlyStop`** (:344-373): lấy mẫu theo đợt (giới hạn đồng thời để bảo vệ gateway) → sau mỗi đợt `isDecisionLocked(soFar, n - drawn, cfg, fusion)` — nếu **không còn kịch bản nào của các phiếu chưa lấy có thể đổi được phán quyết flag/không-flag** thì dừng sớm; cam kết: **không bao giờ TỐN NHIỀU lần gọi hơn lấy đủ n** (:341-342). Lưu ý doc :35-40: `agreement/confidence` chỉ

tính trên các mẫu đã lấy → có thể lệch nhẹ so với lấy đủ n, dù phán quyết flag/không-flag **được bảo đảm giống hệt**.

- **ConsensusVerdict** (:52-61): **samples []** (nguồn gốc cho phân tích đồng thuận của luận văn), **agreement** từ 0 đến 1, **evidenceFusion** {static: leak/clean/ambiguous, dynamic: confirmed/cleared/none}, **overridden** — **heuristic phủ quyết FLAG của consensus**.
- Kết quả FREEZE nhóm 5 (P0 §3): n=50 đảo ngược — single 2.0% flips/F1 0.852 hơn consensus K=3 8.0%/0.793 (McNemar p=0.077) — consensus không được khuyến nghị làm mặc định.

5. Dải borderline — ai được gọi LLM

thresholds.ts:23-29: borderlineLow=0.35, borderlineHigh=0.7 — dải độ tin heuristic mà phán quyết **không rõ/không mơ hồ vẫn coi là borderline** → xứng đáng ý kiến thứ hai từ LLM (consensus). Đây là chiếc van đã gặp suốt chuyến tour: MemHint hầu như không có bundle vào dải → LLM judge 2/17.1K phán quyết; Juliet có 33.6% → LLM thực sự tham gia. Hai cổng precision (§1) chủ động cho ra **likely_false_positive** ở ngoài dải borderline để **không** phải nâng lên LLM.

6. Báo cáo — từ verdict đến 5+3 artifact

- **buildReport** (**reporting.ts:25-67**): tóm tắt = confirmed / likely / falsePositives / totalBytesLost / **toolsUsed** (tập từ verdict.tool + evidence.tool — chính là bằng chứng đường đi của judge!) / durationSec.
- **5 trình dựng chuỗi thuần** (:69-87): **toJson**, **toMarkdown**, **toHtml**, **toSnapshot**, **toCsv** — **"toSnapshot là định dạng chuẩn cho việc máy-so-sánh-máy trong đánh giá thực nghiệm"** (:6) — mọi con số của luận văn đối chiếu qua snapshot, không qua markdown.
- **writeReports** (**reportSink.ts:35-57**): **results/<scanId>/** chứa **report.json/report.md/report.html/snapshot.json/report.csv** + **transcript.json** (tái lập) + **steps.md; metrics.json** (writeScanMetrics :29-33 — phức hợp verdict/độ tin/chi phí — chính là file đã phân tích ở P3 §6); **events.json** ghi **dần dần trong lúc quét** bởi JsonlFileSink (P1 §3).

Tự kiểm P7

1. Rò rỉ động khớp **definitely_lost** cộng mấy điểm, tối đa mấy finding được cộng? (+0.5; 1 finding mạnh nhất — lấy max, không cộng dồn.)
2. Vì sao finding không khớp candidate bị bỏ qua hoàn toàn (chứ không +0.15 như cũ)? (Rò rỉ ở chỗ khác cùng file ≠ bằng chứng cho lệnh cấp phát này — lỗi precision đã sửa.)
3. Hai cổng precision là gì, nguyên tắc chung là gì? (Cổng 1: chạy sạch là bằng chứng tha thứ trừ khi có tín hiệu mạnh; Cổng 2: muốn flag cần tín hiệu mạnh — nguyên tắc: tín hiệu yếu không đủ để khẳng định.)
4. Vì sao cổng 1 cũ sai trên dự án thật? (Chạy "mù" không tham số hầu như không chạm logic ứng dụng → exercised_clean ≠ đã chứng minh sạch; sửa = kéo hasStrongSignal lên trước cả 2 cổng.)
5. Diff của LLM khi nào được dùng? (isDiffApplicable + diffAddsCleanup + diffIsMinimal; sai → diff cố định từ nguồn thay thế.)
6. Sửa SYSTEM_PROMPT có cần tăng số phiên bản cache thủ công không? (Không — bấm prompt là một phần khóa cache, sửa prompt tự làm lệch khóa.)
7. earlyStop bảo đảm gì, không bảo đảm gì? (Phán quyết flag/không-flag giống hệt lấy đủ n và không bao giờ tốn thêm lần gọi; agreement/confidence chỉ tính trên mẫu đã lấy — có thể lệch nhẹ.)
8. **overridden** trong ConsensusVerdict nghĩa là gì? (Heuristic phủ quyết FLAG của consensus — lớp rẻ chống FP có quyền đè lớp đắt.)
9. Định dạng nào là chuẩn để so sánh thực nghiệm? (snapshot.json — máy so máy; markdown/html cho người đọc.)

P8 — Khung đánh giá: corpus gate, determinism gate, ánh xạ FREEZE (1 buổi)

Mục tiêu

Trả lời câu hỏi bảo vệ quan trọng nhất về **độ tin cậy của số liệu**: "vì sao tin được con số trong FREEZE?" Học xong P8 bạn sẽ được cả chuỗi bảo vệ: corpus bị khóa bằng băm nội dung → cửa chặn từ chối chạy trước khi số liệu sinh ra → chấm điểm theo site không cho phép thổi phồng → cấu hình baseline có Zod chặn tổ hợp vô nghĩa → determinism gate chặn cả các cách đạt pass giả → metrics.json ghi rõ nguồn gốc (cờ priced, verdict_tool).

1. Cửa chặn toàn vẹn corpus (`domain/corpusLock.ts`)

Vấn đề gốc (:1-11): trường nguồn gốc từng băm CHỈ `corpus_manifest.json` — một file NGUỒN bị hỏng hoặc bị chế (ví dụ header C++ Juliet không compile được) là **vô hình**. Giải pháp: băm **TẤT CẢ file nguồn** của mọi case.

- **corpusContentHash** (:36-57): từ manifest, với mỗi case duyệt các file nguồn (`listSourceFiles`), tạo `parts = ["caseId/tên_tệp:sha256(file)"]`, **sắp xếp rồi nối** → sha256 → lấy 32 ký tự hex đầu. Sắp xếp để thứ tự duyệt không ảnh hưởng băm (tính cố định). File đọc không được → **sự vắng mặt làm đổi băm — đúng chủ ý** (:52).
- **checkCorpusGate** (:82-91) — 3 tình huống chặn:
 1. Không có lockfile → "run validate-corpus --write-lock";
 2. Lockfile ghi `validated: false` (có case bị cách ly);
 3. **Băm lệch**: băm hiện tại ≠ băm đã khóa. Cả 3 đều làm eval **từ chối chạy**. Lối thoát duy nhất: `--allow-unvalidated` — và TUI hiển thị **thanh cảnh báo thường trực** (`stores/types.ts:111-114`) để "một con số không bao giờ mơ hồ về mức tin cậy của chính nó".
- Lockfile nằm **kế bên** thư mục corpus (`<corpusDir>.lock.json` :61) — ví dụ lockfile MemHint ghi băm `442de35d...` (FREEZE hàng 10).
- **Lưu ý bảo vệ thuật toán**: thuật toán băm **phải giữ nguyên từng byte** so với `scripts/corpus/validate-corpus.ts` (:9-10) — sửa một phía là toàn bộ cửa chặn hỏng.

2. Chấm điểm theo site (`domain/evalScoring.ts`) — không cho thổi phồng

Ground truth 2 tầng (`LabeledCase` :30-49): `flaws[]` + `clean[]` với hàm

- dòng tùy chọn; `allocators/deallocators` theo case ($\approx$ AllocSource/FreeSink của LAMeD); **positive_only** (:58-62) — LAMeD/MemHint chỉ gắn nhãn đúng 1 lỗi trong codebase thật, **mọi thứ khác là CHƯA KIỂM TRA, không phải đã-kiểm-tra-sạch** → `scoreCase` chấm khác nhau cho FP giả định.

2 chế độ (:160-174): `isLineMode` (mọi flaw/clean đều có dòng — kiểu Juliet: cấp phát trong hàm flaw là flaw) so với chế độ theo hàm. **classifyFinding** (:170-174): khớp dòng trước → khớp hàm; chế độ dòng mà không khớp nhãn nào → `'unknown'` (**loại ra, không đoán**) — "cấp phát chưa gắn nhãn là unknown, không phải đoán mò".

Gộp theo site — chống thổi phồng (:182-247):

- **siteKey** (:188-190): **file:dòng** (chế độ dòng) hoặc tên hàm — các finding **cùng site gộp thành 1 mẫu** → scanner báo 2 lần (hoặc 2 công cụ cùng báo) **không thể thổi TP/FP** — "một lỗi thật là một ô trong bảng nhầm lẫn".
- Flagged thẳng khi trùng site (:233-237) — mang confidence của nó.
- **Lỗi bị bỏ sót hoàn toàn** → **tạo mẫu FN tổng hợp** (:249-254) — **siteId** khớp đúng khóa mà finding lẽ ra tạo → **thiết kế cho kiểm định ghép cặp**.
- **ExtraFinding** (:259+): finding bị flag **không khớp nhãn nào** → được ghi lại, báo cáo, **không được chấm** (:218-220) — nhớ **positive_only** quyết cách đếm chúng.

Sample (`common/analysis/metrics.ts:9-22`): `{actual, predicted, confidence, siteId}` — `siteId = <caseId>::<siteKey>` toàn cục → **2 lần chạy (single so với consensus) khớp nhau theo site cho kiểm định McNemar ghép cặp** (FREEZE nhóm 5 dùng đúng cái này: 205 site ghép cặp).

Nguồn gốc trong finding (:66-88): **verdict_tool** ("dùng để chứng minh run llm_assisted THẬT SỰ dùng LLM — tính toàn vẹn của nguồn gốc"), **dynamic_coverage** (chẩn đoán, không dùng để chấm).

3. Cấu hình baseline dạng khai báo (`domain/baselineConfig.ts`)

5 trục năng lực (:24-35): `static / dynamic / planner / tool_selector / fusion` — mỗi YAML trong `configs/baselines/B*.yaml` là một vector; B6a với B6b tách riêng planner/tool_selector (2 trục ĐỘC LẬP — :8-11).

Zod superRefine chặn tổ hợp vô nghĩa (:42-82):

- `!static && !dynamic` → "nothing to detect";
- `tool_selector && !fusion / planner && !fusion` → giai đoạn LLM mà thiếu LLM;
- `runs > 1` khi **fusion** tắt → lỗi: "run cố định, lấy phương sai vô nghĩa" (:73-81).
- **Trùng id giữa 2 file** → throw (:112-117) — "một va chạm id sẽ âm thầm làm mất một hàng trong bảng ablation".

Đây là mẫu thiết kế "cấu hình định nghĩa thí nghiệm → ép buộc kiểu im lặng làm hỏng kết quả" (:13-15).

4. Determinism gate — Tier-1 được CI chặn

README + `docs/EVALUATION.md` §7 (two-tier): `no_llm` bắt buộc **trùng từng byte** giữa 2 lần chạy (thư mục kết quả riêng, cùng cấu hình) — bảng Juliet n=50: **TP29 FP7 FN3 TN38**. `scripts/determinism-gate.sh` + `assert-determinism.ts` **chủ động loại 2 cách đạt pass giả** từng gặp thật khi phát triển:

1. **So sánh bản thân với chính mình qua va chạm timestamp** — tưởng là so 2 lần chạy thực chất so chính nó;
2. **Run toàn lỗi** — 2 lần cùng toàn lỗi thì "giống nhau" nhưng vô nghĩa, mạo danh là cố định.

Bài học thiết kế: cửa chặn phải **từ chối cả những cách ĐẠT pass giả** chứ không chỉ khẳng định "bằng nhau". `llm_assisted` thì Tier-2: `--runs N` lấy trung bình ± độ lệch + `verdict-stability.ts` (tỉ lệ lật verdict từng case).

5. Eval CLI (`evaluation/flags.ts`) — mặt trận tùy chọn

Điểm chọn lọc từ `CliFlags` (:9-62): `--stratify` (khóa mặc định `functionalVariant`), `--runs N`, `--resume` (driver MemHint dùng — P8.5 §4), `--allow-unvalidated`, **rào chi phí**: `--max-case-ms`, `--max-case-cost-usd`, `--max-consecutive-errors`, `--pause-on-quota-exhausted` (dừng thay vì im lặng rơi

về heuristic khi hết hạn ngạch); `--judge-cache`; `--set-endpoint provider.field=value`; chế độ quét: `--baseline B1,B6a` / `--all-baselines`. `--ingest-kind` phân biệt corpus khi tự nạp.

6. `metrics.json` — số mô tả từng lần quét (`domain/scanMetrics.ts`)

Đây là file đã phân rã token ở P3 §6. `computeScanMetrics` (:79-131) đếm: verdicts / `root_cause_counts` (patternType) / `verdict_tool_counts` (nguồn gốc đường đi của judge) / `dynamic_coverage_counts` / confidence min/mean/max / token / thời lượng / `mcp_calls`. `cost_usd` chỉ có khi đã cấu hình bảng giá — comment :69: "`undefined when no price configured — never $0 as a stand-in`" — không bao giờ giả \$0. Cờ `priced: boolean` ghi rõ tình trạng.

7. Kiểm chứng bộ nhận diện allocator (`scripts/validate-allocator-profile.ts`)

Đây là **nguồn của FREEZE nhóm 6** (cJSON: allocators P35%/R92% F1 0.51, deallocators P67%/R100% F1 0.80 — củng cố khẳng định của bài báo MemHint):

- Chạy `profileAllocators` (LLM) trên **1 case đại diện mỗi dự án** (API allocator là cấp dự án :68-72), so với **ground truth đã đóng bằng PROJECT_ALLOCATORS/DEALLOCATORS** từ `scripts/lamed/ingest.ts`.
- Thiết kế tinh tế (:4-11): đo chất lượng bộ phát hiện LLM, nhưng **eval rò rỉ vẫn dùng danh sách đóng băng** → việc thay danh sách cứng bằng LLM discovery không làm eval mất tính cố định. LLM được đo ở tầng profiler, bị khóa ở tầng eval.

8. Ánh xạ FREEZE ↔ code (nhớ để đối chiếu khi bị hỏi)

FREEZE hàng/nhóm	Nguồn code/số
Hàng 1 sweep 9-baseline	<code>configs/baselines/B*.yaml</code> + <code>scripts/run-baselines.ts</code> + <code>evaluation/</code> quét; F1 từ <code>metrics.ts</code> qua bộ chấm snapshot
Hàng 3 llm_assisted toàn corpus	<code>evaluate-corpus.ts</code> <code>--runs</code> ; đường đi judge = <code>verdict_tool_counts</code>
Hàng 4 LAMeD 4 lần chạy	chấm <code>positive_only</code> (:58-62)
Hàng 5 consensus ablation	<code>judgeByConsensus</code> + <code>Sample.siteId</code> → McNemar 205 site
Hàng 6 allocator profile	<code>validate-allocator-profile.ts</code>
Hàng 9 Juliet Tier-1	determinism gate + bảng TP29/FP7/FN3/TN38
Hàng 10 MemHint	corpus lock <code>442de35d...</code> + <code>metrics.json</code> (mcpCalls, token)

Tự kiểm P8

- Vì sao bấm từng vô hình với file nguồn hồng, sửa thế nào? (*Bấm chỉ manifest → sửa: bấm mọi file nguồn, sắp-xếp-ghép; file đọc không được cũng làm đổi bấm.*)
- 3 tình huống chặn của corpus gate? (*Không lockfile / validated:false / bấm lệch; lối thoát duy nhất --allow-unvalidated kèm thanh cảnh báo.*)

3. Scanner flag cùng site 3 lần (2 công cụ) thì bảng nhằm lần đếm mấy? (1 — gộp site; flagged thẳng + mang confidence.)
4. Finding flag không khớp nhãn nào đi đâu? (ExtraFinding — ghi lại, không chấm; cách đếm phụ thuộc positive_only.)
5. `positive_only` đổi ngữ nghĩa gì? (Flag ngoài nhãn = FP giả định vì corpus chỉ gắn nhãn 1 lỗi, còn lại CHƯA KIỂM TRA chứ không phải đã-kiểm-tra-sạch.)
6. 2 cách đặt pass giả mà determinism gate chặn? (Tự-so-sánh qua va chạm timestamp; run toàn lỗi mạo danh cố định.)
7. Tổ hợp baseline nào bị Zod chặn? (Không có gì để phát hiện; planner/ tool_selector thiếu fusion; runs>1 khi fusion tắt; trùng id.)
8. Vì sao `cost_usd` là undefined thay vì \$0? (Chưa cấu hình bảng giá → không đoán — thay \$0 sẽ làm méo mọi phân tích chi phí.)
9. LLM phát hiện allocator được đo ở đâu mà eval vẫn cố định? (validate-allocator-profile đo riêng; eval dùng danh sách đóng băng.)

P8.5 — Dữ liệu đánh giá: 3 corpus đến từ đâu, dựng thế nào, chạy thế nào (1 buổi)

Mục tiêu

Trả lời trọn bộ câu "dataset từ đâu, dựng thế nào, chạy thế nào": nguồn gốc 3 corpus (tải về / tái lập từ paper / khai quật lịch sử git), chuỗi khởi tạo (manifest → kiểm chứng → lockfile → gate), và sổ tay chạy thực tế gồm cả các sự cố (OOM, máy ảo khởi động lại) đã xử lý thế nào. Sau P8.5 bạn tự dựng lại được toàn bộ chuỗi từ một máy sạch đến các con số FREEZE.

1. Ba corpus — ba nguồn gốc khác nhau (bảng tổng)

	Juliet CWE-401	LAMeD	MemHint
Nguồn	TẢI: NIST SARD v1.3 (public domain, file zip)	Tái lập từ bảng paper (EASE 2025, đã bình duyệt)	Khai quật git-log độc lập (memhint_bugs.json, đã commit)
Bộ nạp	<code>scripts/juliet/ingest.ts</code>	<code>scripts/lamed/ingest.ts</code>	<code>scripts/memhint/ingest.ts</code>
Quy mô	1,658 case (tổng hợp)	41 case / 7 dự án	19 case / 6 dự án (trong 8 dự án mục tiêu của MemHint, bỏ FFmpeg + linux/staging)
Nhãn	Suy ra từ quy ước đặt tên (<code>bad/good</code> + FLAW_RE)	Từ bảng paper (file + target_function + commit)	1 lỗi/case từ commit sửa lỗi thật, <code>github_url</code> kiểm chứng từng mục
Kiểm chứng nhãn	cổng compile + nhãn mềm	<code>--strict-labels</code> + <code>--skip-compile</code>	<code>--strict-labels</code> + <code>--skip-compile</code>
Thư mục	<code>demo/juliet_cwe401</code>	<code>demo/lamed</code>	<code>demo/memhint</code>

Điểm cần thuộc về MemHint (`scripts/memhint/ingest.ts:8-13` — comment mở đầu nói thẳng): corpus này **KHÔNG phải tái lập 54 lỗi của bài báo** — danh sách đó **không được công bố ở đâu cả** (repo chỉ có công cụ + lệnh build; Bảng I chỉ là số tổng hợp; `memory_safety_bugs.json` trong repo là kết quả CHẠY CỦA CÔNG CỤ, không phải chuẩn mực). Nguồn sự thật là `memhint_bugs.json`: **khai quật lịch sử git** trên chính upstream từng dự án (khớp mẫu tin nhắn commit + đọc diff bằng tay để xác nhận đúng lỗi thiếu-free CWE-401, không phải refactor hay loại lỗi khác) — mỗi mục đều kiểm chứng được độc lập.

2. Bộ nạp — cơ chế từng loại

2a. Juliet (`scripts/juliet/ingest.ts`)

- Đọc cây NIST đã giải nén (`--juliet <thư_mục_gốc>`), gom testcase theo tên gốc (tách file bad/good), giới hạn qua `--limit`.
- **Copy các file hỗ trợ**: bộ lỗi (`std_testcase.h`, `io.c...`) + quét `#include` để lấy header cục bộ (:216-238) — mỗi case tự khai báo những gì nó phụ thuộc.
- Sinh **Makefile từ mẫu** + `build_command: make clean && make — scripts/juliet/repair-makefiles.ts` là công cụ sửa một lần cho corpus đã dựng (sửa main trùng, dọn sản phẩm build rác).
- **Nhấn suy ra: FLAW_RE** (comment **POTENTIAL FLAW**) + hàm định nghĩa trong file `bad*/good*` — dữ liệu tổng hợp nên quy ước đặt tên chính là chuẩn mực.
- Nguồn gốc từng file: `{name, sha256}` ghi vào `_provenance` → lockfile chứng minh các byte đến từ cây NIST đã xác minh.

2b. LAMeD (`scripts/lamed/ingest.ts`)

- Phân tích bảng paper: `parseGithubRef (github.com/org/repo/(tree|commit)/sha` → repo + sha) → checkout **commit cha** (trạng thái TRƯỚC khi sửa — lỗi còn sống) → chụp bản chụp mỗi case.
- **Bẫy phân tích cần thuộc** (:78-100): LAMeD dùng `;` quá tải — vừa là dấu tách THAM SỐ trong chữ ký, vừa là dấu tách GIỮA CÁC HÀM (`main; static void f(void)`), thậm chí cắt cụt giữa chữ ký → `splitTopLevelSemicolons` **đếm độ sâu ngoặc** (không tách trong ngoặc); `bareFunctionName` bỏ qua macro (callee đầu tiên có chữ thường).
- **Sự trung thực có chủ đích** (:112-124): 6/41 mục `target_function` rỗng → lỗi cấp file (function '') — **được ghi lại trung thực qua `fileLevelOnly`, không bị bỏ đi** (chỉ chấm được ở chế độ dòng mà LAMeD không có — sự thật được ghi, không biến mất).
- Allocator/Deallocator theo dự án ($\approx$ AllocSource/FreeSink của LAMeD).

2c. MemHint (`scripts/memhint/ingest.ts`)

- `--benchmark` mặc định `demo/memhint/memhint_bugs.json`; `--manifest-only` (chỉ ghi manifest, không clone); `--clones` thư mục cache clone.
- **Dựng vật chất** (:162-180): `git clone --filter=blob:none` (đu lịch sử để giải mọi SHA, không tải blob thừa) → `checkout parent_commit` (mã **TRƯỚC** khi sửa — lỗi còn sống) → `cpSync` cây làm việc (bỏ `.git`) thành bản chụp ổn định theo từng case.
- **PROJECT_ALLOCATORS** (:69-87) — kiểm chứng trực tiếp bằng grep trên header của dự án, không đoán; **câu chuyện curl cần thuộc**: curl-8.19.0 (phiên bản MemHint ghim) **đổi tên API cấp phát cấp nguồn thành `curlx_*`** qua macro trong `lib/curl_setup.h` — chỉ thêm `Curl_c*` (tên mà bản chụp curl cũ hơn của LAMeD dùng) sẽ **âm thầm bỏ sót mọi lệnh cấp phát `curlx_*`** → giữ CẢ HAI.
- **PROJECT_BUILD_COMMANDS** (:109-122): điều chỉnh từ `proj_build_command.json` của chính repo **MemHint** (được tác giả xác minh); các bước `can_error:true` được bọc (`cmd; true`); `autoreconf -fi && ./configure / ./Configure` được thêm cho checkout mới (JSON gốc giả định cây đã cấu hình) — ghi rõ là thử nghiệm tốt nhất, cần xác minh trong bước kiểm tra build.

- Mỗi case ghi nguồn gốc `_memhint: {githubUrl, fixCommit, parentCommit, notes}` — bộ chấm không đọc, nhưng nhấn truy vết được.

3. Manifest v2 + schema khi chạy

- Schema: `memory-leak-corpus/v2`; zod khi chạy `LabeledManifestSchema` (`packages/common/src/validation/corpus-manifest.schema.ts`) — trước khi có schema này, manifest hồng chạm tới bộ chấm và **lệch số liệu một cách im lặng** (:2-7). Refine chặn case **không chấm được** (không flaws, không clean, không count :48-52); `.passthrough()` dung nạp các trường nguồn gốc thêm (`_memhint, _lamed`).
- Kiểm chứng + lockfile** (`scripts/corpus/validate-corpus.ts --write-lock`, qua wrapper `evaluation/ingestRunners.ts:37-42`):
 - Cờ theo từng corpus có lý do**: Juliet = công compile từng file + nhấn mềm; `LAMeD/MemHint = --skip-compile + --strict-labels` — comment :30-36 nói rõ: `clang -fsyntax-only` từng file là công cụ SAI cho dự án đa-file autotools/CMake; nhấn là thẩm quyền duy nhất (positive-only, không có fallback theo quy ước đặt tên kiểu Juliet).
 - Lockfile `corpus-lock/v1: {contentHash, validated, summary{total, clean, warned, quarantined}, toolVersions{clang}, ingestCommit}` — gate ở P8 §1 chỉ mở khi `validated &&` bấm khớp.
- Catalog + wizard** (`evaluation/corpusCatalog.ts`): 3 `KNOWN_CORPORA` + `discoverCorpora()` (corpus tạm có manifest dưới `demo/`); 3 trạng thái (`not-ingested / unvalidated-has-manifest / validated`) — đây là chỗ duy nhất liệt kê cả corpus CHƯA nạp (wizard TUI dùng nó để đề xuất nạp). Wizard có tự tải zip Juliet **khi người dùng chủ động chọn** (mặc định tắt, `downloadFile curl :49-54`).

4. Chiến lược chạy thực tế — số tay đã đi qua thật

4a. Trình tự chuẩn (mọi corpus)

```

nạp → validate-corpus --write-lock → checkCorpusGate (từ chối nếu lệch)
→ chạy no_llm (mốc cố định, 1 lần)
→ chạy llm_assisted xN (--runs; phương sai mean±std)
→ verdict-stability (tỉ lệ lật) → snapshot.json = chuẩn so sánh

```

4b. MemHint — driver + xử lý sự cố (chapter4:316)

`scripts/memhint-eval-driver.sh` trên WSL2, commit `30e04cb1c`:

- Redis đẩy analyzer tĩnh vào dài OOM (khoảng 14GB RSS)** → driver thiết kế **8 lần thử lại** với `--resume`; máy ảo **khởi động lại giữa run** làm mất lần thử 1 → chốt ở **lần thử 3/8**; các sản phẩm **sha256-xác minh trùng từng byte qua lần khởi động lại** (đây là lý do tên thư mục giữ mốc `2026-08-28` của lần đầu).
- Đối chiếu mốc**: lần đầu chỉ chấm được 11/19 case → mốc R 42.3% (11/26, 8 case mất vì OOM); run chốt khôi phục 19/19 → R 46.2%. Vì `no_llm` trùng từng byte, 11 case cũ **bắt buộc** cho kết quả y hệt → +1 TP chắc chắn đến từ case khôi phục, không phải thay đổi code — **tính cố định biến một sự cố thành một bằng chứng**.
- Rào chặn của eval CLI: `--max-case-cost-usd, --max-case-ms, --pause-on-quota-exhausted` (dừng thay vì im lặng rơi về heuristic khi hết hạn ngạch), `--resume`.

4c. Juliet — quét 9-baseline

9 file YAML `configs/baselines/B1-B7` × toàn corpus × 3 lần chạy mỗi cấu hình (fusion), một commit engine `5eec8b1`, tổng \$75.78 — FREEZE hàng 1. `n=50 stratified` là mẫu nhanh để so cấu hình với nhau; toàn corpus là bài kiểm tra khái quát hóa.

Tự kiểm P8.5

1. Vì sao corpus MemHint KHÔNG phải "tái lập" của bài báo? (*Danh sách 54 lỗi không được công bố — Bảng I chỉ số tổng hợp; JSON trong repo là kết quả công cụ, không phải chuẩn mực; corpus = dựng lại độc lập từ commit sửa lỗi.*)
2. Checkout commit nào để dựng case, vì sao? (`parent_commit` — mã TRƯỚC khi sửa, lỗi còn sống.)
3. Sự cố curl allocator là gì? (`curl-8.19.0` đổi tên `Curl_c*` → `curlx_*` qua macro — chỉ giữ tên cũ = âm thầm bỏ sót toàn bộ lệnh cấp phát mới; sửa: giữ cả 2.)
4. Vì sao LAMeD/MemHint kiểm chứng bằng `--skip-compile --strict-labels`? (`clang -fsyntax-only` từng file sai cho đa-file autotools/CMake; nhấn là thẩm quyền — không có fallback theo tên như Juliet.)
5. 6/41 mục LAMeD target_function rỗng xử lý thế nào? (Lỗi cấp file, gán cờ `fileLevelOnly` — ghi trung thực, không bỏ.)
6. Máy ào khởi động lại giữa run — vì sao +1 TP vẫn đáng tin? (`no_llm` trùng từng byte → 11 case cũ bắt buộc y hệt; +1 chắc chắn từ case khôi phục.)
7. Chỗ duy nhất liệt kê corpus chưa nạp là gì? (`corpusCatalog.ts` — `KNOWN_CORPORA` gộp `discoverCorpora`; wizard dùng để đề xuất nạp.)

P9 — Chuẩn bị bảo vệ: 20 câu hỏi + câu chuyện số liệu (1 buổi)

Mục tiêu

Phần cuối: (a) **20 câu hỏi bảo vệ dễ gặp nhất** kèm khung trả lời 30–60 giây mỗi câu, (b) **cách kể câu chuyện số liệu** — mỗi con số FREEZE được kể như một mạch lý do, không phải bảng học thuộc. Mọi câu trả lời dưới đây đều có chỗ dựa trong TOUR P0–P8.5 — ôn xong phần nào, câu đó tự tin hơn.

1. Câu chuyện số liệu — kể theo mạch, không theo bảng

1a. Mạch "hệ thống hoạt động" (con số chính)

"Hệ được đánh giá trên 3 corpus với hai chế độ. **Juliet 1,658 case** (đã kiểm chứng, băm `f578c3ee`): quét 9-cấu hình cho thấy cấu hình mạnh nhất **B6a (planner + công thức ghim + LLM judge)** đạt **F1 0.863±0.001 / MCC 0.790 trên toàn corpus** với chi phí \$6.63 — vượt static-only B1 (0.612) và Clang Static Analyzer (~0.76 trên `n=50`). **n=50 stratified**: B6a F1 **0.938**. Khoảng cách `n=50` với toàn corpus là một hạn chế được nói thẳng: 2 nhóm yếu (C++ new/delete R 16.7%, malloc P 36.7%) bị mẫu stratified pha loãng. **LAMeD 41 case** (dự án thật, positive-only): `llm_assisted` 3 lần chạy TP15/FP0/FN35 R 30% — LLM judge chạy 47–149 lần/lượt nhưng không lật phán quyết nào. **MemHint 19 case**: `no_llm` và `llm_assisted` ×3 **đều TP12/FP0/FN14, F1 0.632±0.000** — độ lệch bằng 0 qua 3 lần chạy."

1b. Mạch "consensus bị đảo ngược — và đó là một kết quả khoa học"

"Đóng góp đầu tiên của tôi là consensus judge (lấy k mẫu). Thí nghiệm đầu (30 case, 2 lần lặp lại độc lập) xác nhận: lật 6.7% so với single 13.3–26.7%. Nhưng khi mở rộng lên **n=50 stratified (2026-08-**

19), kết quả **ĐÀO NGƯỢC**: single 2.0% lật / F1 0.852 so với consensus K=3 8.0% / 0.793 (McNemar 205 site ghép cặp, $p=0.077$ — nghiêng về single, chưa đủ tin cậy thống kê). Tôi báo cáo cả hai và **khuyến nghị single làm mặc định** — FREEZE nhóm 5 giữ kết quả cũ kèm con trỏ sang kết quả mới. Đây là minh họa cho nguyên tắc của tôi: số đo xong là số bị đóng băng, kể cả khi nó bác bỏ chính đóng góp của tôi."

1c. Mạch "chi phí và vì sao hybrid judge là câu trả lời"

"Trên MemHint, 3 lần chạy llm_assisted tốn khoảng 44.3M token — phân rã cho thấy gần như toàn bộ thuộc **Stage A** (sub-agent thu thập bằng chứng: 88.9K lời gọi MCP mỗi lần chạy, ~4.7K calls/case), trong khi **LLM judge chỉ chạy 2 lần trong 3 lần chạy** vì hybrid judge để heuristic xử lý khoảng 17.1K phán quyết mỗi lần và chỉ 2 bundle borderline. Kết quả: F1 không đổi so với no_llm — corpus không borderline thì chi phí LLM là thuần túy. Thiết kế hybrid chính là để chi phí LLM **tỉ lệ với số bundle thực sự cần LLM**; hướng phát triển tiếp theo là cổng chọn lọc cho Stage A — chỉ mở Stage A cho bundle mà bằng chứng cố định để lại mơ hồ — tương tự mẫu **dynamic: selective** đã có sẵn."

1d. Ba câu chuyện gỡ lỗi (mỗi cái = một bài học thiết kế)

- Trùng bundleId** (P2 §3): id cắt 6+10 ký tự khiến 622 candidates của **curl_1098e104** rơi còn 65 bundle (cùng đầu đường dẫn + cùng đuôi tên allocator). Bài học: *độ dài phần băm phải tỉ lệ với độ dài khóa; băm toàn bộ khóa.*
- Cổng chạy-sạch "mù"** (P7 §1): trên dự án thật, buildTarget→IsanRun chạy chương trình không tham số → **exercised_clean** nghĩa là "chưa chạy tới", không phải "đã chứng minh sạch" → cổng cũ để lần chạy động đề bằng chứng tĩnh tốt. Bài học: *ý nghĩa của mỗi trạng thái coverage phải ghi rõ ở hợp đồng và judge phải tôn trọng đúng ý nghĩa đó.*
- Quá tải MCP 8→4** (P4 §8): 6 case đồng thời × 8 = 48 lời gọi MCP đồng thời → 9/19 case MemHint timeout. Sửa ở 2 đầu: giới hạn phía người gọi + pool worker phía máy chủ + trần heap từng worker (P5 §2a). Bài học: *tạo áp lực tải cần kiểm soát ở cả 2 phía.*

2. 20 câu hỏi bảo vệ + khung trả lời

Kiến trúc (A):

- Vì sao chọn MCP thay vì gRPC/REST?* → MCP là giao thức của việc agent gọi công cụ; gRPC hết người dùng khi bỏ đường web (P0 §1). Schema một catalog **tool-catalog** hai đầu dùng chung (P5 §1).
- LLM có thể bịa bằng chứng không?* → Không: lớp bọc capture ghi mọi kết quả công cụ cố định vào store, "wrapper is the sole source"; LLM chỉ lái, không tự ghi phán quyết (P3 §3, P3 §5).
- Vì sao giai đoạn động là công thức chứ không cho LLM chọn công cụ?* → Cô lập tính không-lặp-lại vào judge (đóng góp #3); worker LLM chỉ dự phòng khi không có buildCommand và tool_selector bật (P3 §5).
- Kiến trúc "framework-free" nghĩa là gì, vì sao?* → Tự viết queryLoop + callModel + compaction (P4); điểm tiềm phụ thuộc cho phép test không cần mạng; không bị ràng buộc API framework nào.
- Một lần quét đi qua những giai đoạn nào?* → 7 giai đoạn runScan: preflight → discovery → [enrichment | dynamic] → investigation → judging → reporting (P2).

Kỹ thuật (T): 6. *Khả năng lặp lại bảo đảm thế nào?* → 2 tầng: no_llm trùng từng byte (now có thể thay + công thức khóa + determinism gate chặn 2 cách pass giả) so với llm_assisted dạng phân phối; cửa băm corpus (P8 §1, §4). 7. *Cache có phục vụ dữ liệu cũ không?* → 3 tầng đều có trần và có phạm vi: theo từng lần quét (P2), dedup theo tham-số-trừ-content (P3), LRU+đĩa khóa theo băm nội dung (P5) — khóa gồm cả allocators. 8. *Chạy code không tin tưởng an toàn thế nào?* → WORKSPACE_ROOT 4 lớp (resolve/tiền-tổ-các-ký/realpath/từ-

chối-symlink-ra-ngoài), không mạng + giới hạn bộ nhớ/tiến trình, tham số docker dạng mảng, đường binary phải resolve + nằm trong phạm vi; rủi ro còn lại nói thẳng: build script không tin tưởng (P6). 9. *Compaction là gì?* → Cắt tia cố định kết quả công cụ cũ thành placeholder tại chỗ — không tốn lần gọi LLM, không vỡ cặp tool_result (P4 §7). 10. *macOS/Windows chạy phần động được không?* → macOS: đổi leak→address + build trong Docker; Valgrind chỉ Linux, luôn qua Docker (P6 §2c).

Judge (J): 11. *Heuristic chấm thế nào?* → Bảng điểm tín hiệu (P7 §1), ngưỡng đóng băng 0.7/0.4, 2 đường rút ngắn FP (freedViaCallee/Caller), 2 công precision yêu cầu tín hiệu mạnh. 12. *Khi nào LLM judge tham gia?* → Chỉ bundle borderline (độ tin heuristic trong dải 0.35–0.7, P7 §5) + không bị 2 đường rút ngắn chặn. 13. *Consensus có tốt hơn single không?* → Đo cho thấy không trên n=50 — bị đảo ngược, báo cáo cả 2, khuyến nghị single (P9 §1b). 14. *LLM trả JSON bản thì sao?* → parseVerdict trích object khỏi văn bản, Zod kiểm, ghi lý do hỏng; cache verdict khóa theo băm prompt (P7 §3). 15. *Tin heuristic hơn hay LLM hơn?* → Heuristic quyết phần không-borderline; LLM chỉ ý kiến thứ hai cho borderline; heuristic có quyền phủ quyết cả FLAG của consensus (**overridden**, P7 §4).

Đánh giá (E): 16. *Chấm điểm thế nào, có thổi phồng được không?* → Gộp theo site, tạo mẫu FN cho lỗi bị sót, ExtraFinding ghi-lại-không-chấm, positive_only cho corpus chỉ-gắn-1-nhãn (P8 §2). 17. *Đo trên corpus gì, tin được không?* → 3 corpus đã kiểm chứng, lockfile băm mọi file nguồn; MemHint băm **442de35d**, Juliet **f578c3ee** (P8 §1). 18. *Chi phí hệ thống?* → B6a \$6.63 (Juliet toàn corpus); MemHint 44.3M token/3 lần chạy — Stage A chiếm gần hết, judge gần 0; cost_usd chỉ khi có bảng giá (không bao giờ thay \$0) (P3 §6, P8 §6). 19. *Hạn chế lớn nhất?* → Khái quát hóa trên toàn corpus (F1 0.612 tĩnh) + nhóm C++ new/delete yếu + Stage A đi vòng quá nhiều (4.7K calls/case) — cả 3 đều báo cáo thẳng, không giấu (P9 §1a). 20. *Hướng phát triển?* → Cổng chọn lọc cho Stage A (chỉ LLM cho phần mơ hồ), trần lượt gọi, đầu ra gọn cho công cụ, tận dụng cache ngữ cảnh; dựa trên số MemHint: giảm chi phí không mất F1 vì ΔF1=0 đã đo (P3 §6).

3. Danh sách vật chất ngày bảo vệ

- ☐ Kể mạch 1a–1d không cần slide (luyện từng mạch một lần)
- ☐ Mở được sản phẩm thật nếu hội đồng hỏi: **results/memhint-llm-assisted-2026-08-28-run2/cases/freerdp_9fc23ad2.json** (đường đi judge {heuristic:1920, llm:2})
- ☐ **RESULTS-FREEZE.md** 10/10 hàng in sẵn 1 bản
- ☐ PDF 104 trang (Lời cảm ơn đã điền) + slides 20 trang
- ☐ Biết vị trí code trả lời câu A1–T10 (TOUR P0–P6 là bản đồ)
- ☐ Đọc lại THESIS.md + CONTRIBUTION.md trước ngày bảo vệ

Kết chuyến tour

P0–P9 (kèm P8.5) khép vòng: **toàn cảnh** → **lối vào** → **trục chính** → **bộ não LLM** → **hai bộ phận phân tích** → **phán quyết** → **phòng thí nghiệm** → **phòng thủ**. Hệ bây giờ được nắm từ **cli.ts** tới **snapshot.json**, từ cache parse sha1 đến phủ quyết consensus — từng góc ngách đều có file:dòng để đối chiếu. Nếu code thay đổi sau này, sửa đúng phần TOUR tương ứng — danh sách mục ở đầu file là bản đồ bảo trì của chính TOUR.